

嵌入式操作系统

Embedded Operating System

信息与软件工程学院

桑楠

sn@uestc.edu.cn

2026年2月

第三部分

嵌入式操作系统基础

—— 嵌入式软件体系结构

核心内容

- 体系结构的概念
- 软件体系结构的作用
- 软件体系结构的发展历程
- 通用的体系结构
- 基于模型软件开发
- 嵌入式软件体系结构

为何要了解 软件体系结构

对现代软件系统的理解



体系结构的概念

- 目前尚没有共识的定义
- 许多专家学者从不同角度和不同侧面对软件体系结构进行了刻画

来自建筑业的灵感

■ 砖瓦房的建造



来自建筑业的灵感 (2)

■ 现代建造



框架

Dewayne Perry和Alex Wolf定义

- 具有一定形式的结构化元素，即构件的集合，包括处理构件、数据构件和连接构件。
 - 处理构件负责对数据进行加工
 - 数据构件是被加工的信息
 - 连接构件把体系结构的不同部分组组合连接起来。
- 该定义注重区分处理构件、数据构件和连接构件。其思想应用于其他定义中

Kruchten定义

软件体系结构有四个角度

连接

➤ **概念角度**：描述系统的主要构件及它们之间的关系；

数据

➤ **模块角度**：包含功能分解与层次结构；

➤ **运行角度**：描述了一个系统的动态结构；

处理

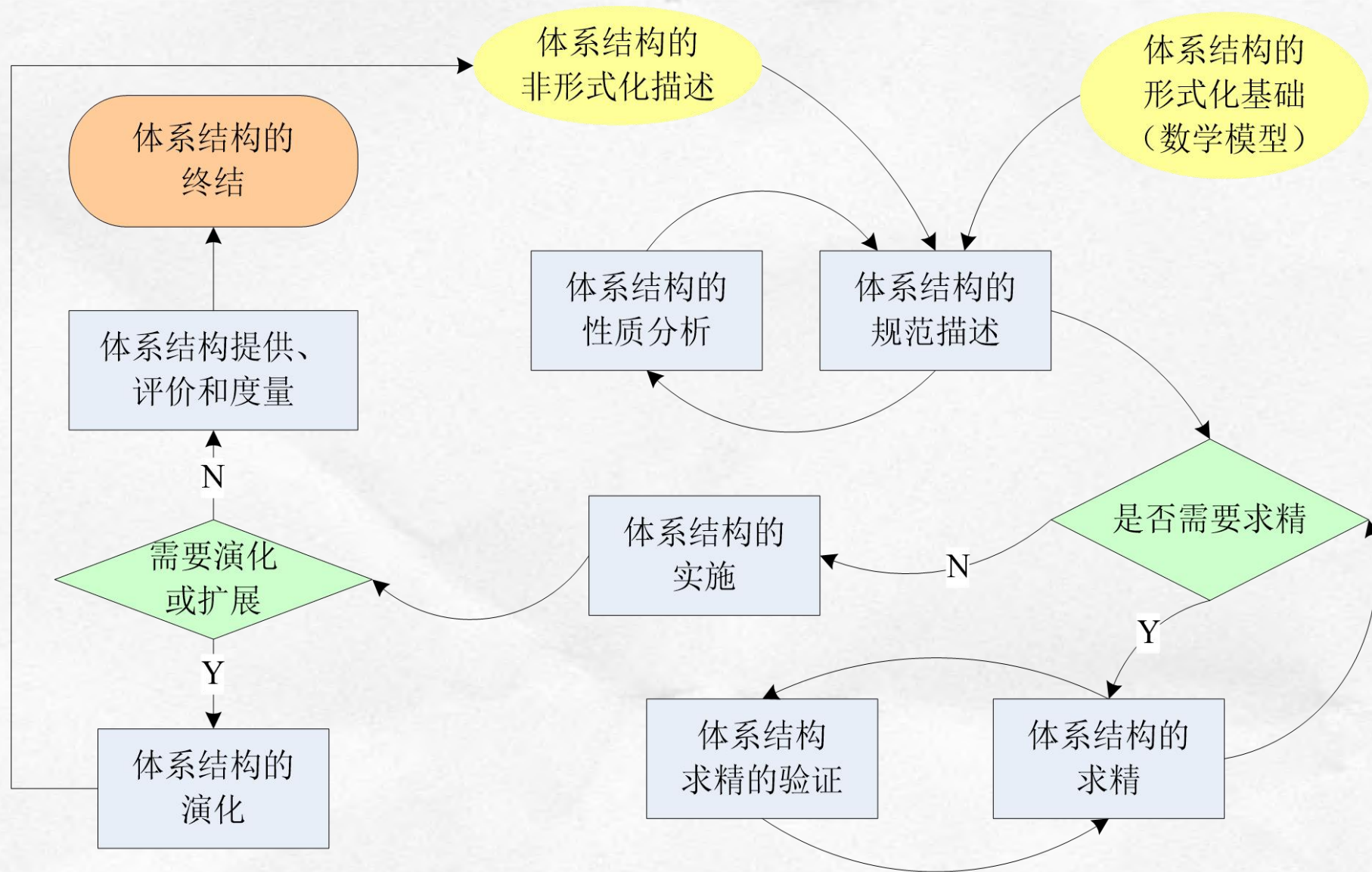
➤ **代码角度**：描述了各种代码和库函数在开发环境中的组织。

处理

Bass, Clements和Kazman定义

- 一个程序或计算机系统的**软件体系结构**包括
 - 一个或一组软件构件、软件构件的外部可见特性及其相互关系。
 - 其中，"软件外部的可见特性"是指软件构件提供的服务、性能、特性、错误处理、共享资源使用等

软件体系结构的生命周期



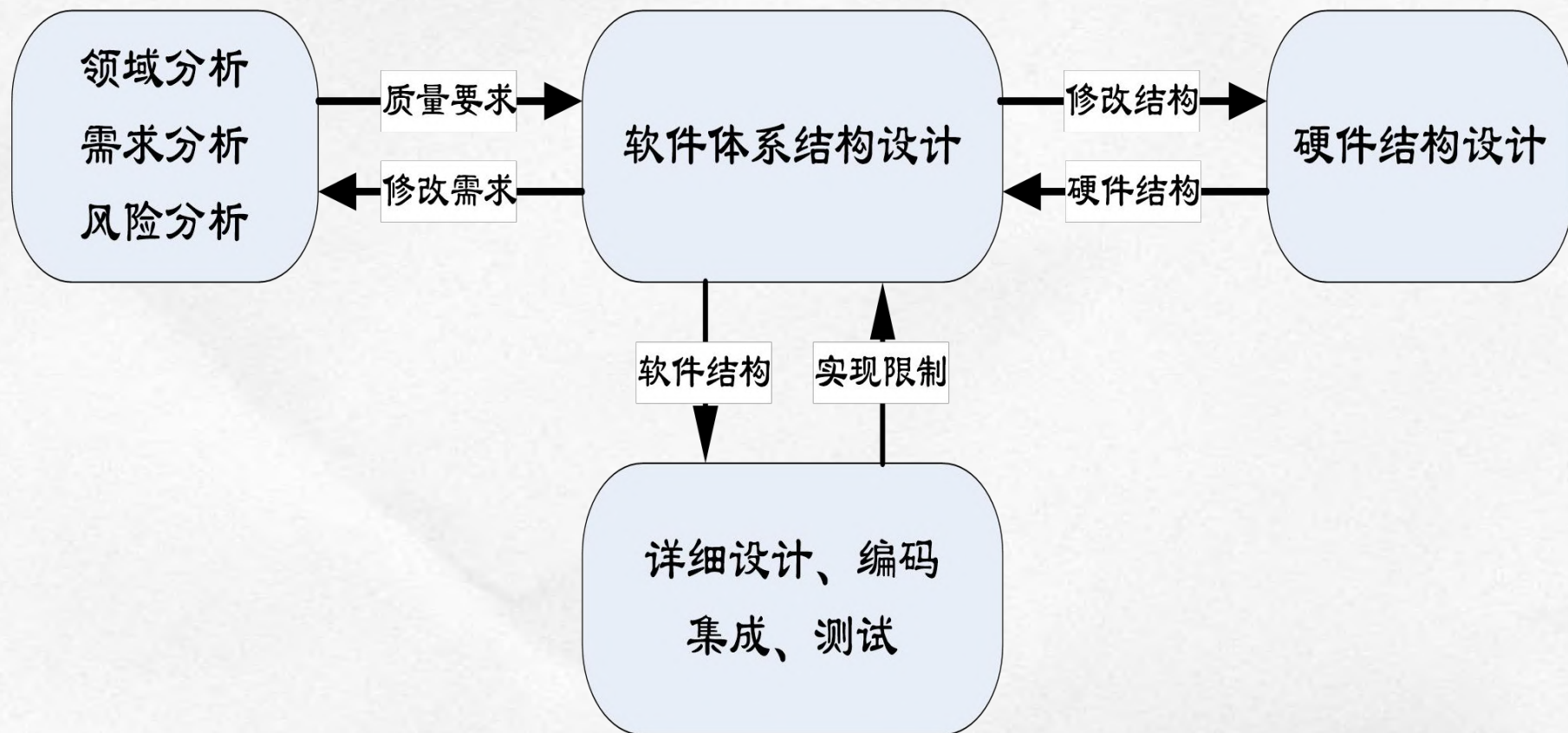
软件体系结构的发展方向

- **各种ADLs（结构描述语言）之间的信息互换**：用一种公共形式把各种语言综合起来，使得能够交换各种体系结构描述信息 → **体系结构融合**
- **设计工具和环境**：用工具实现体系结构的描述和分析，以实现阶段成果的自动转换
- **体系结构再工程**：从遗留系统软件代码和系统中抽取结构信息，经过描述、统一、抽象、一般化与实例化等处理，总结出系统的体系结构

软件体系结构

作用

设计计划



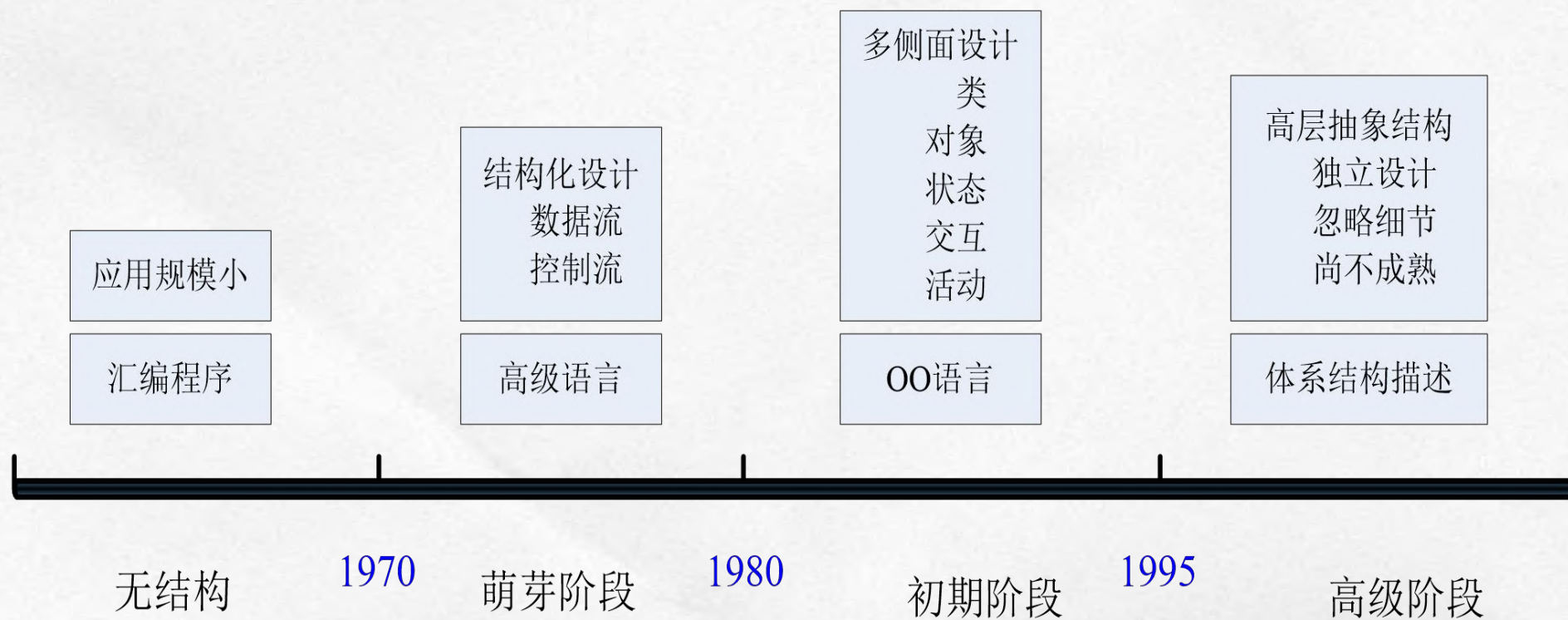
软件体系结构与其他开发任务的关系

帮助管理复杂系统的抽象框架

- 任务的实时性软件体系结构独立于任何一个现实系统平台，它是一个抽象：现实系统中的许多细节被提炼并封装成软件体系结构中的元素。
- 软件体系结构对管理的另一辅助作用是便于分析矛盾需求之间的平衡点。

软件体系结构 ★ 发展历程

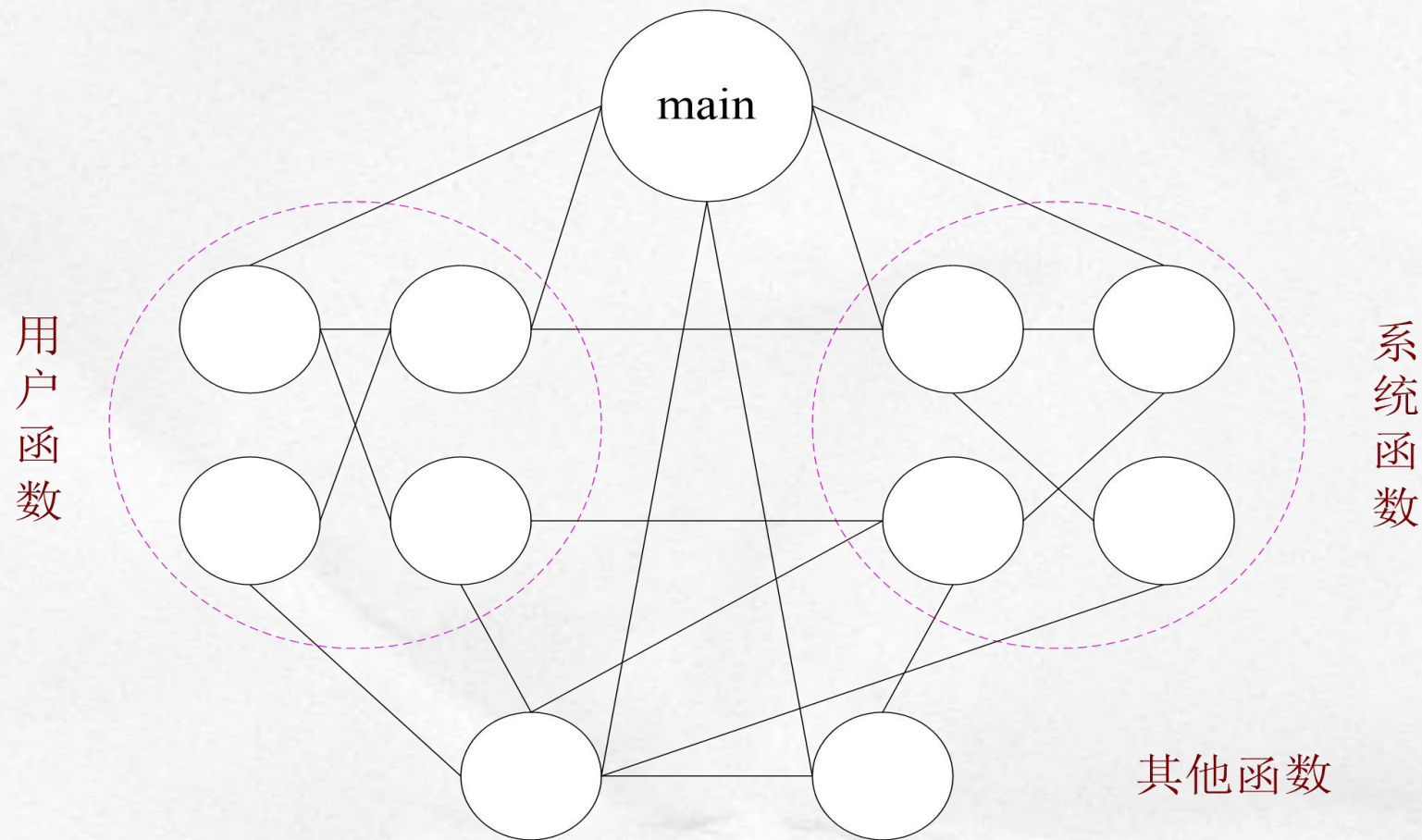
示意



软件体系结构

通用模型

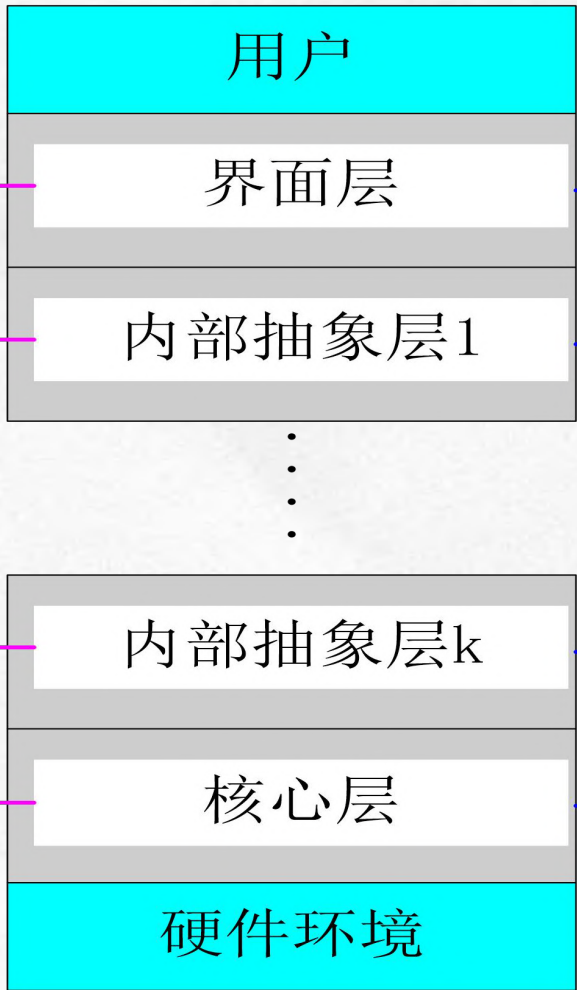
整体结构 — 无结构模型



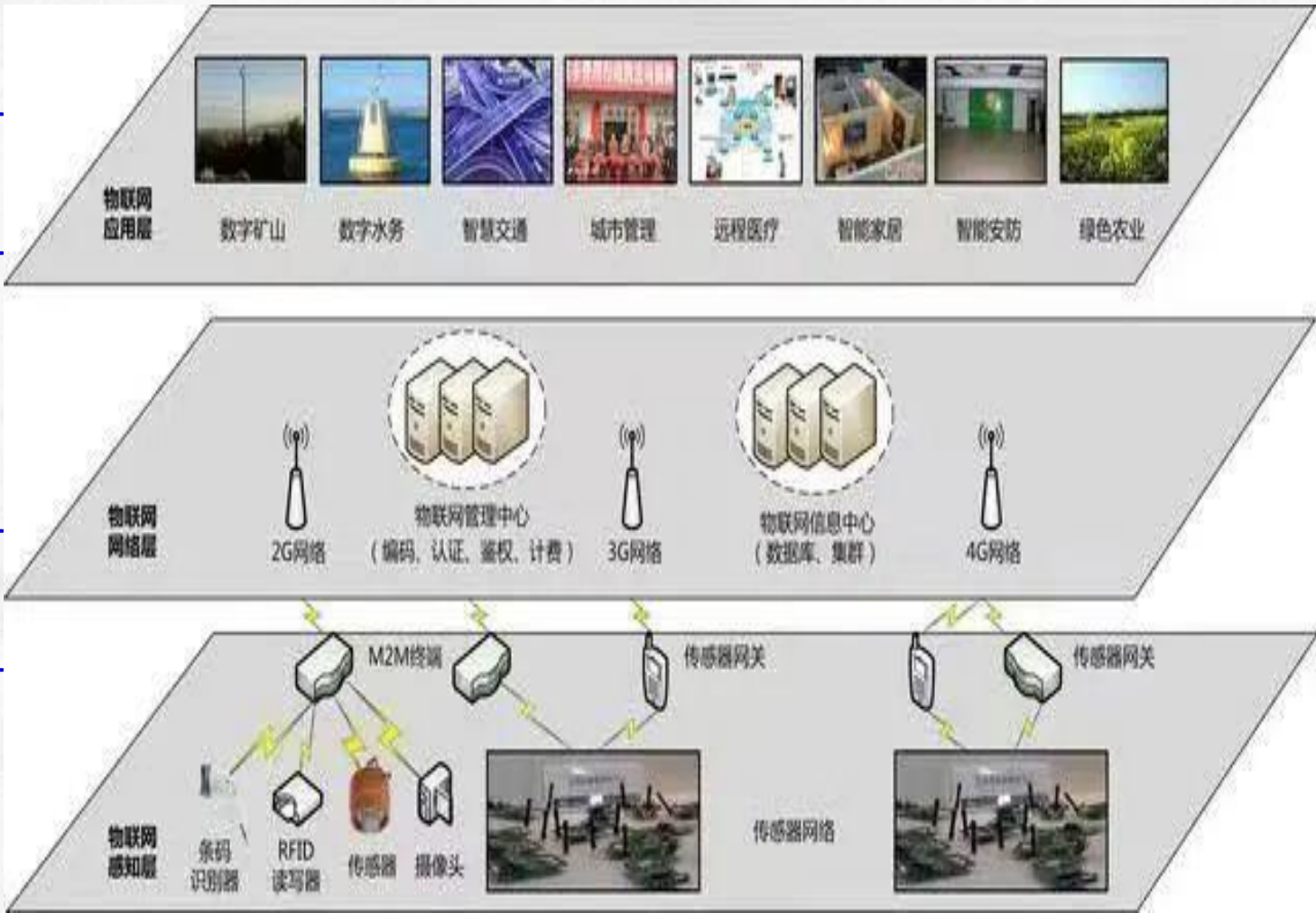
整体结构系统示意模型

层次结构

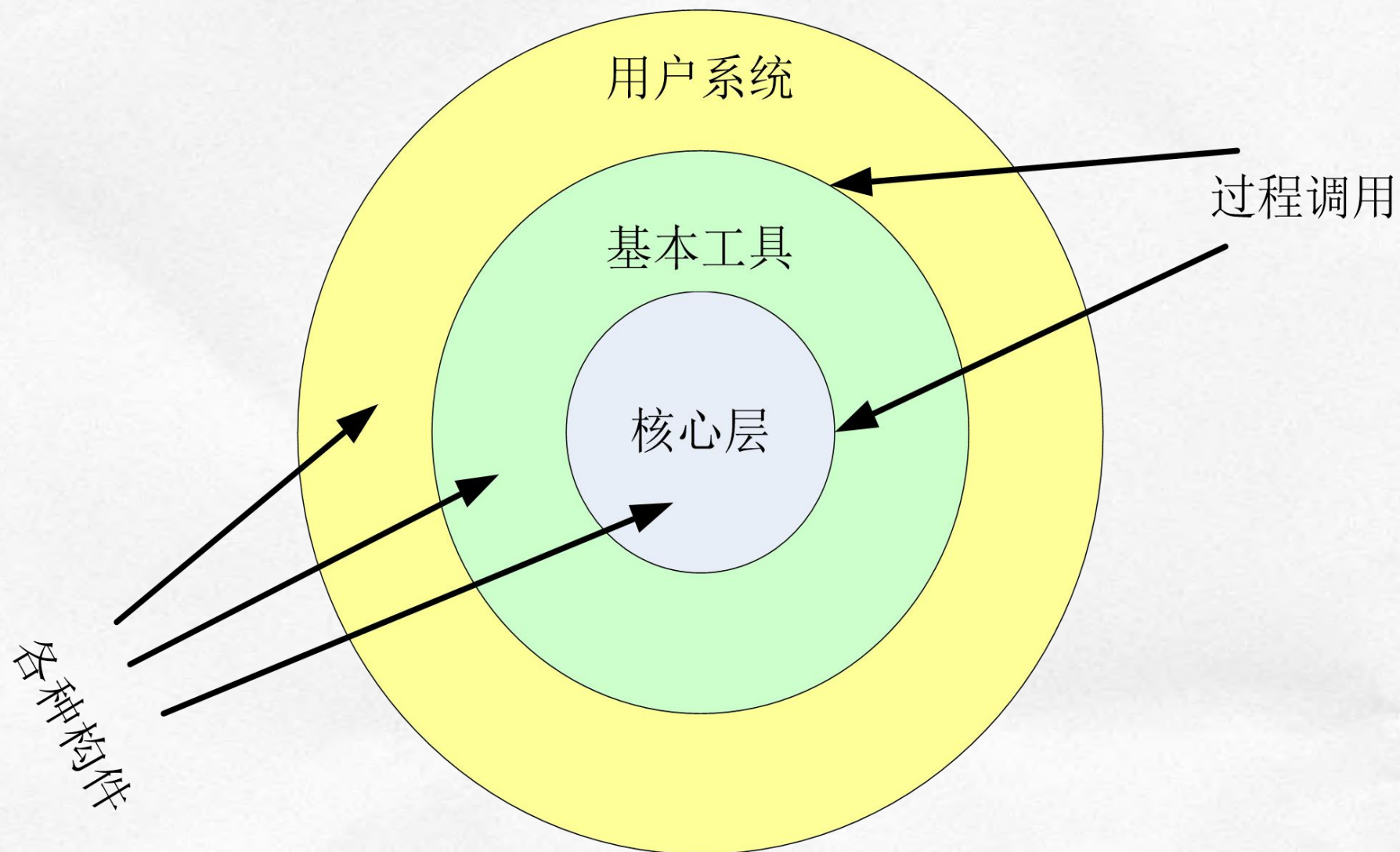
由各层元素组成



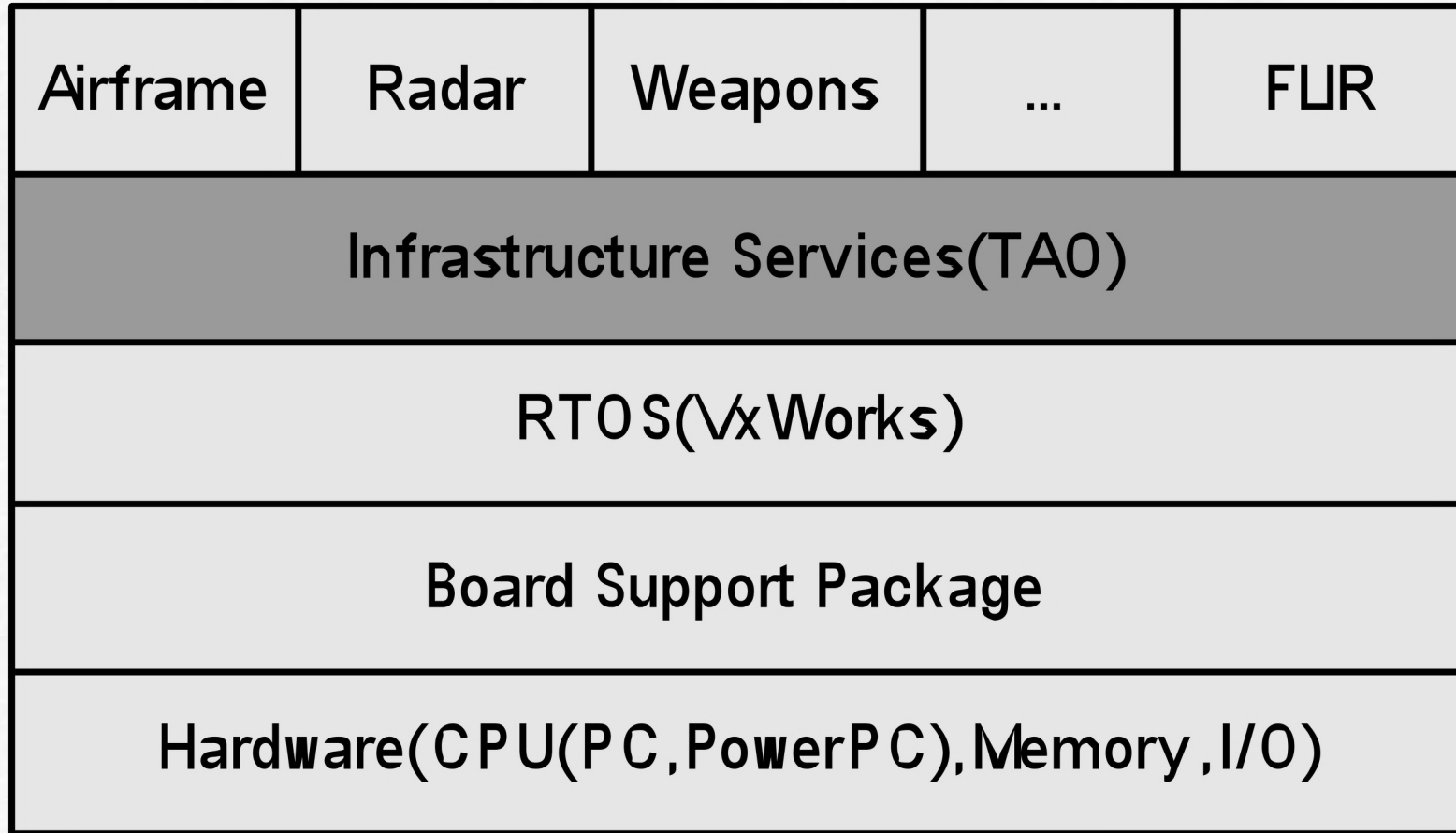
层次结构系统模型



层次结构 (2) — 另一种体现

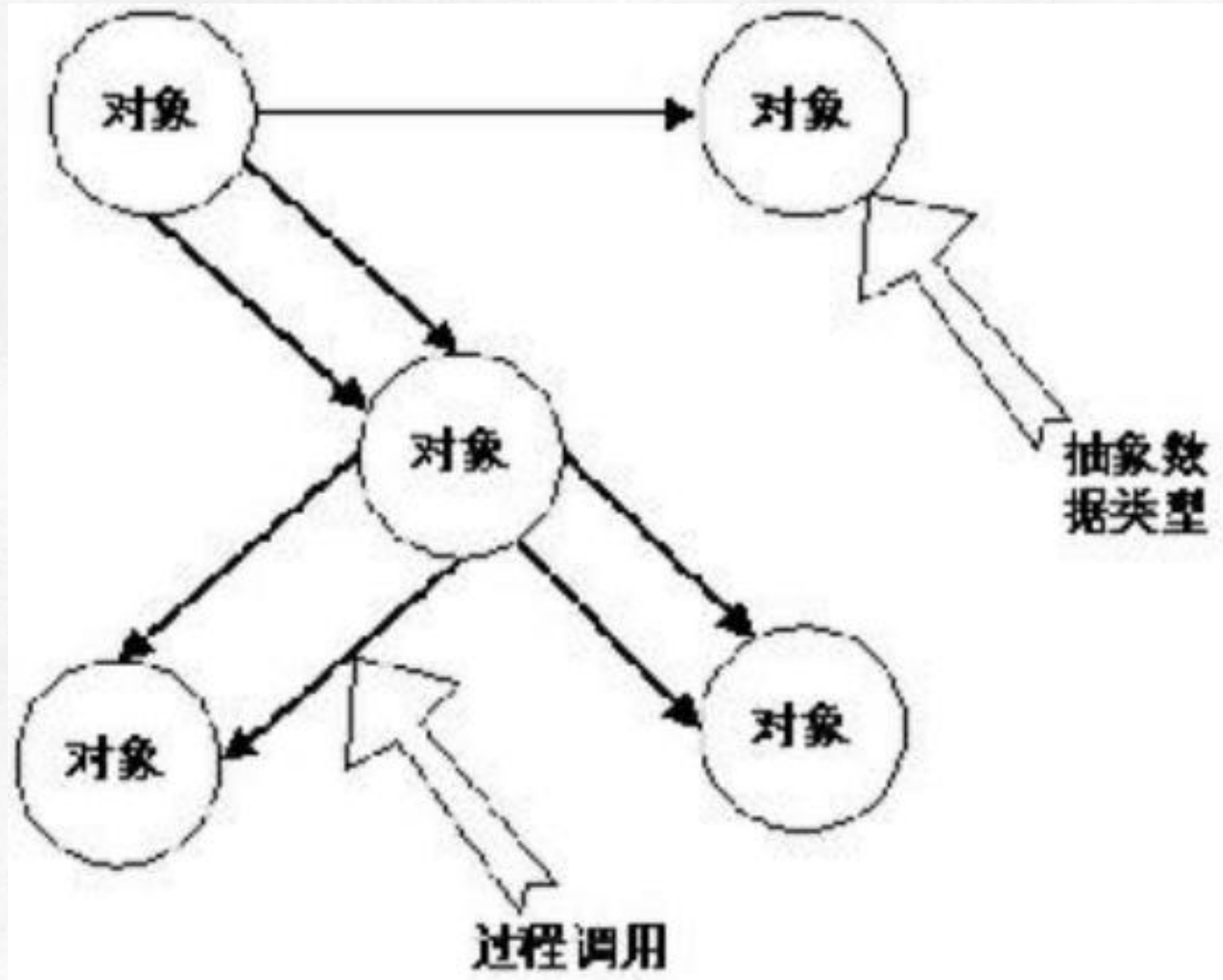


层次结构 (3) — 示例

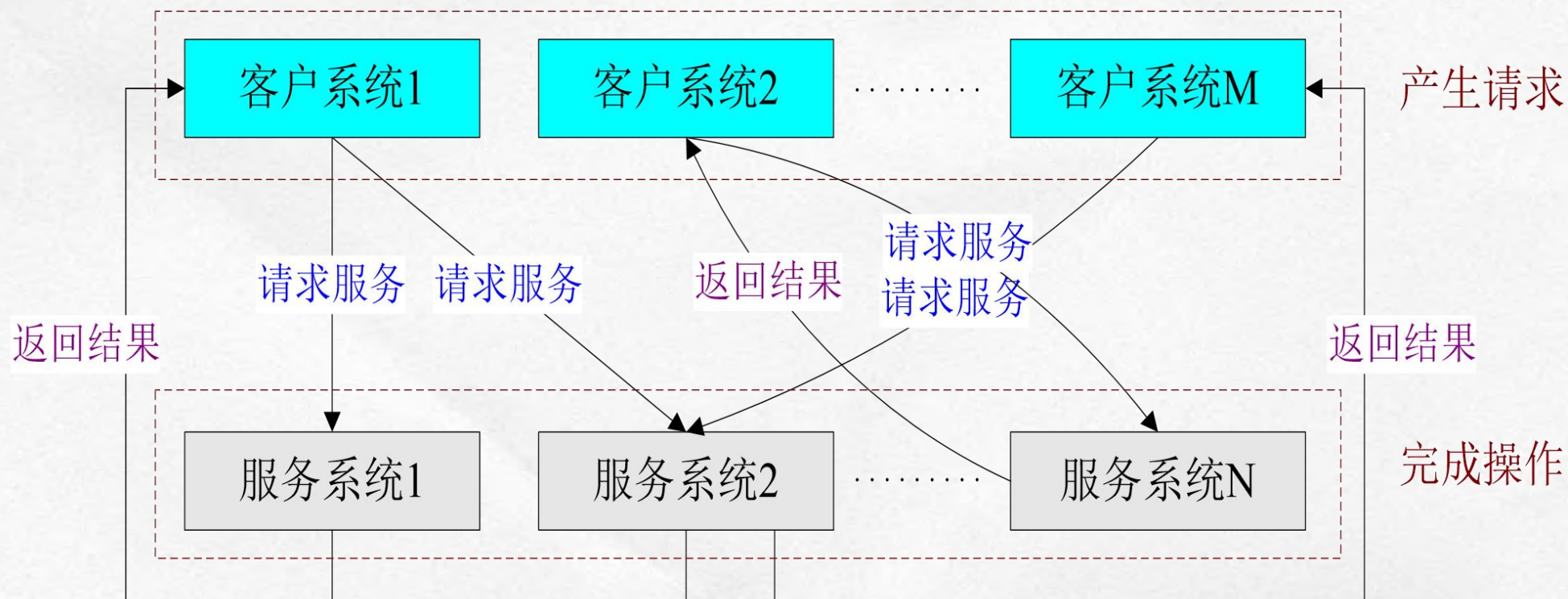


OFP (飞控程序)

对象结构模型 —— 抽象



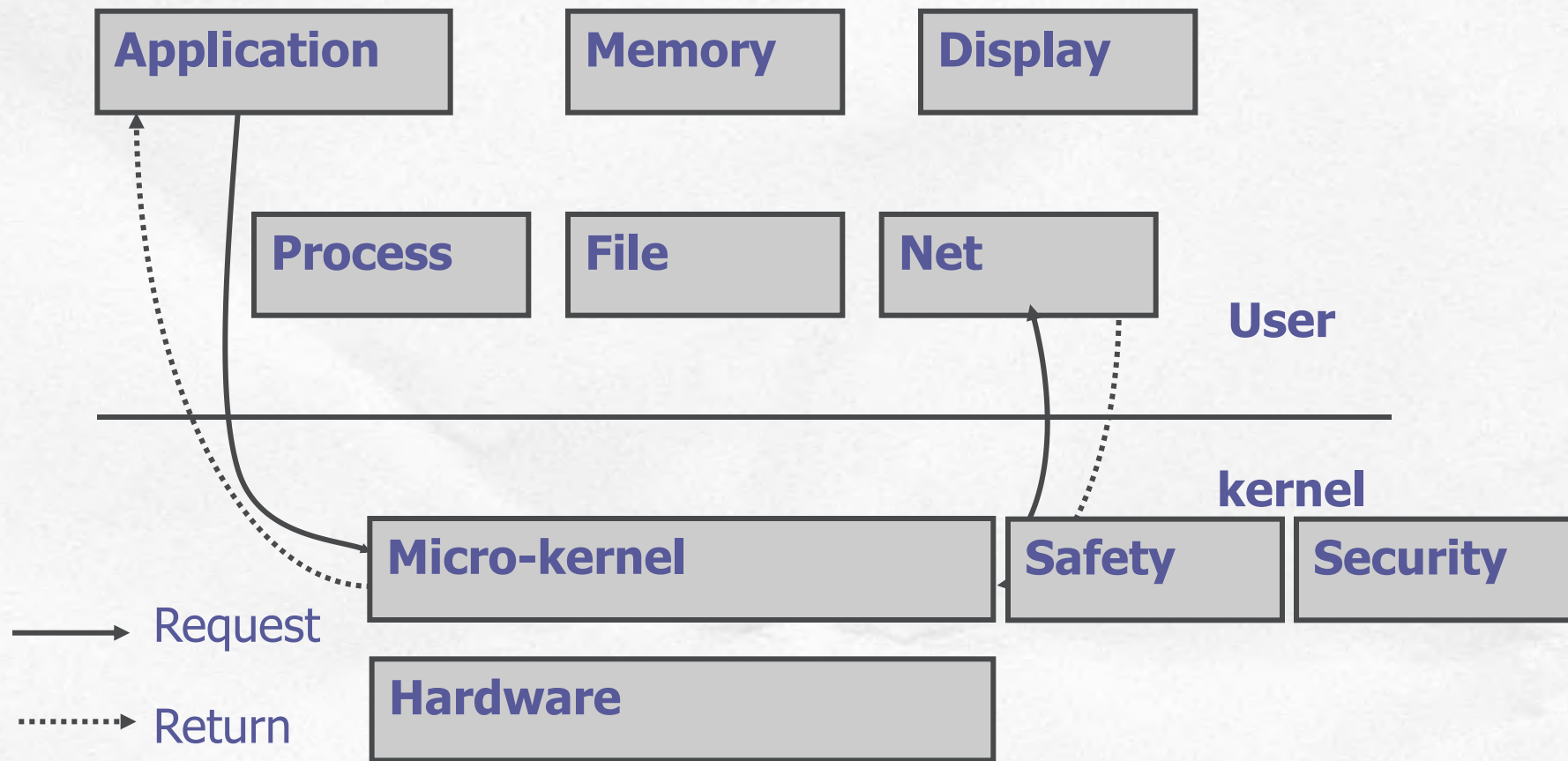
客户机/服务器结构模型



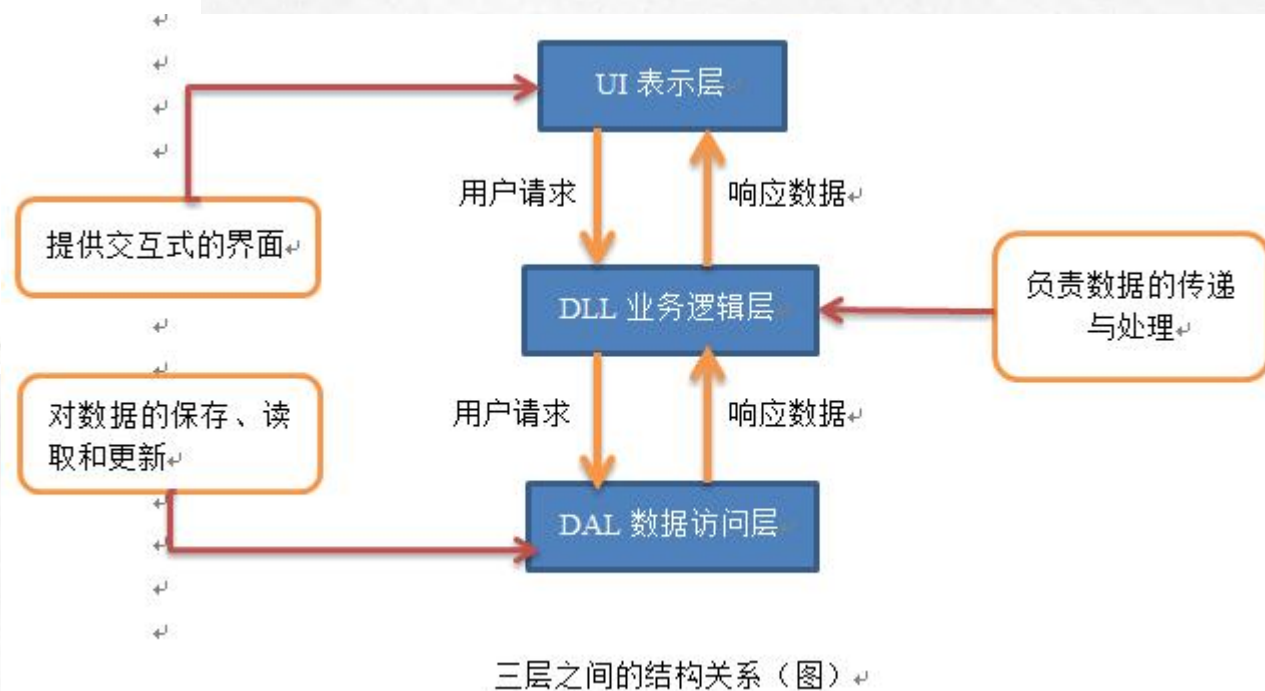
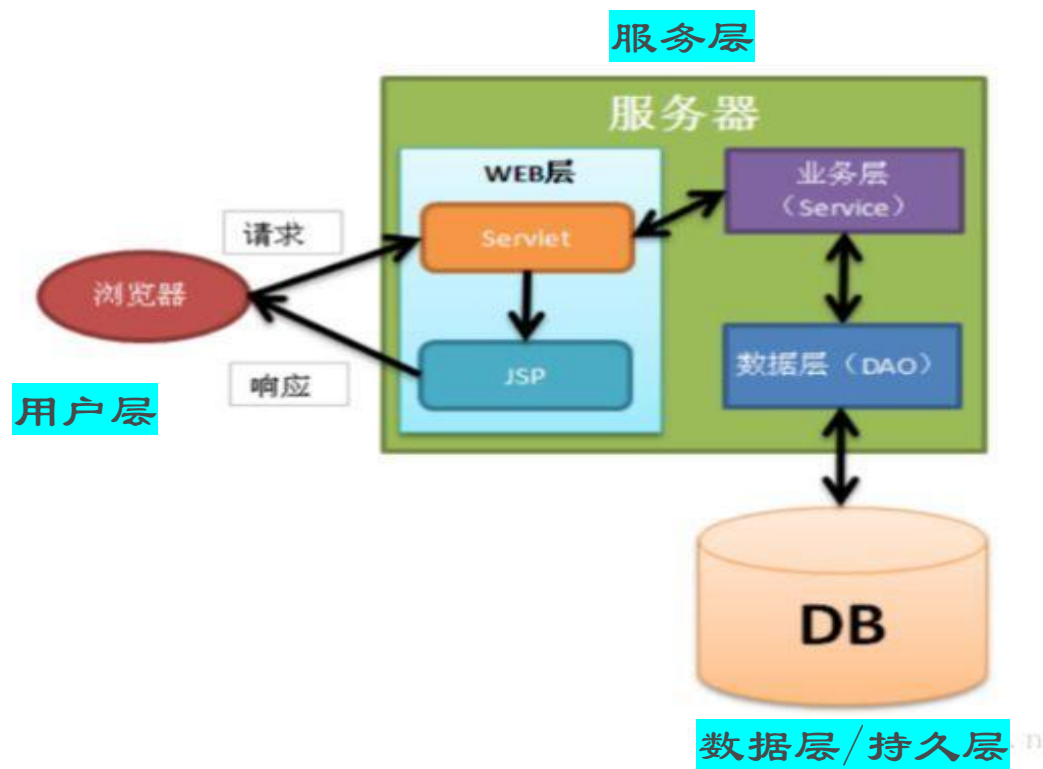
客户机 / 服务器体系结构

客户机/服务器结构模型 (1)

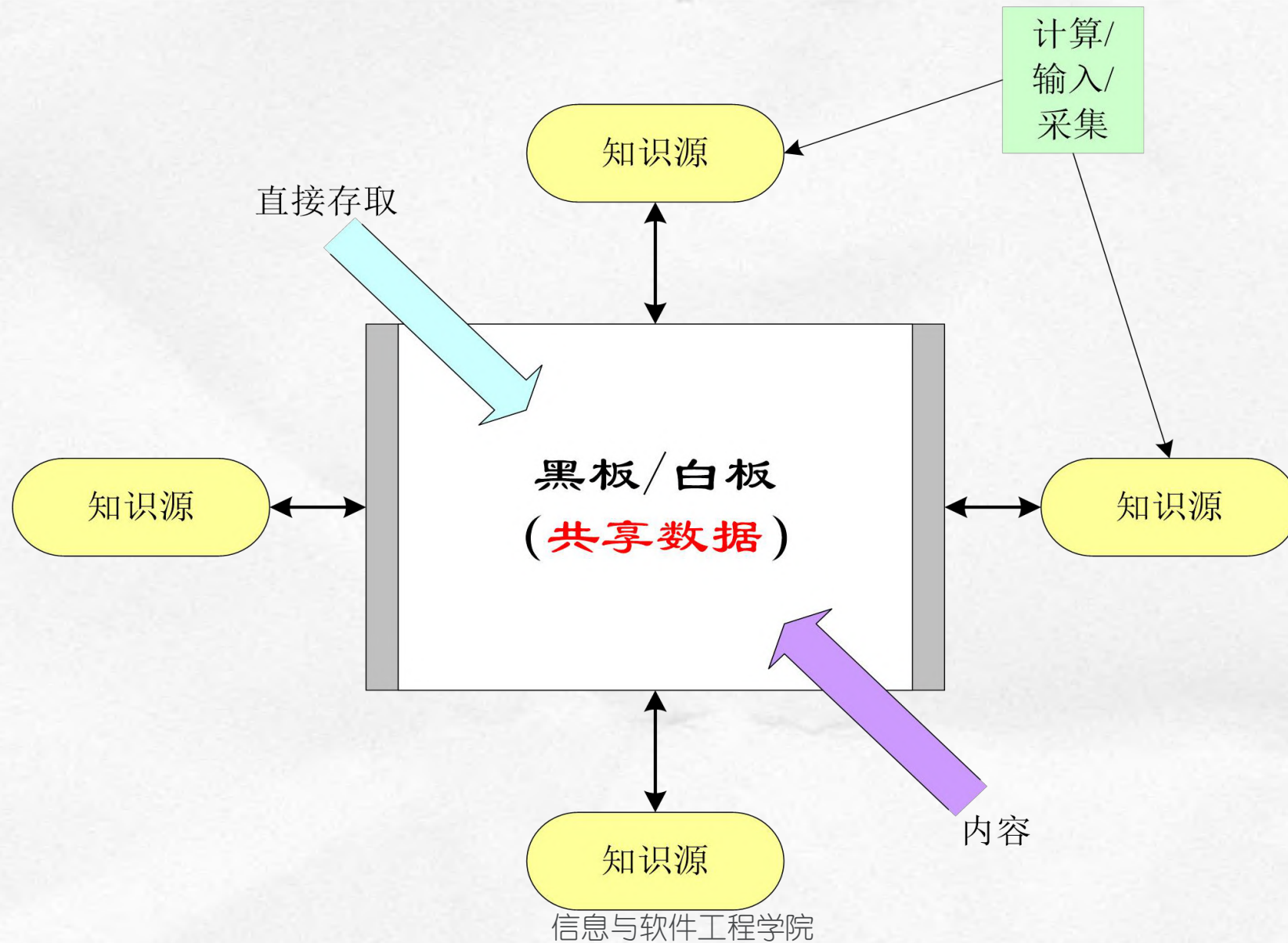
■ 安全OS —— 微内核



三层体系架构



黑板模型

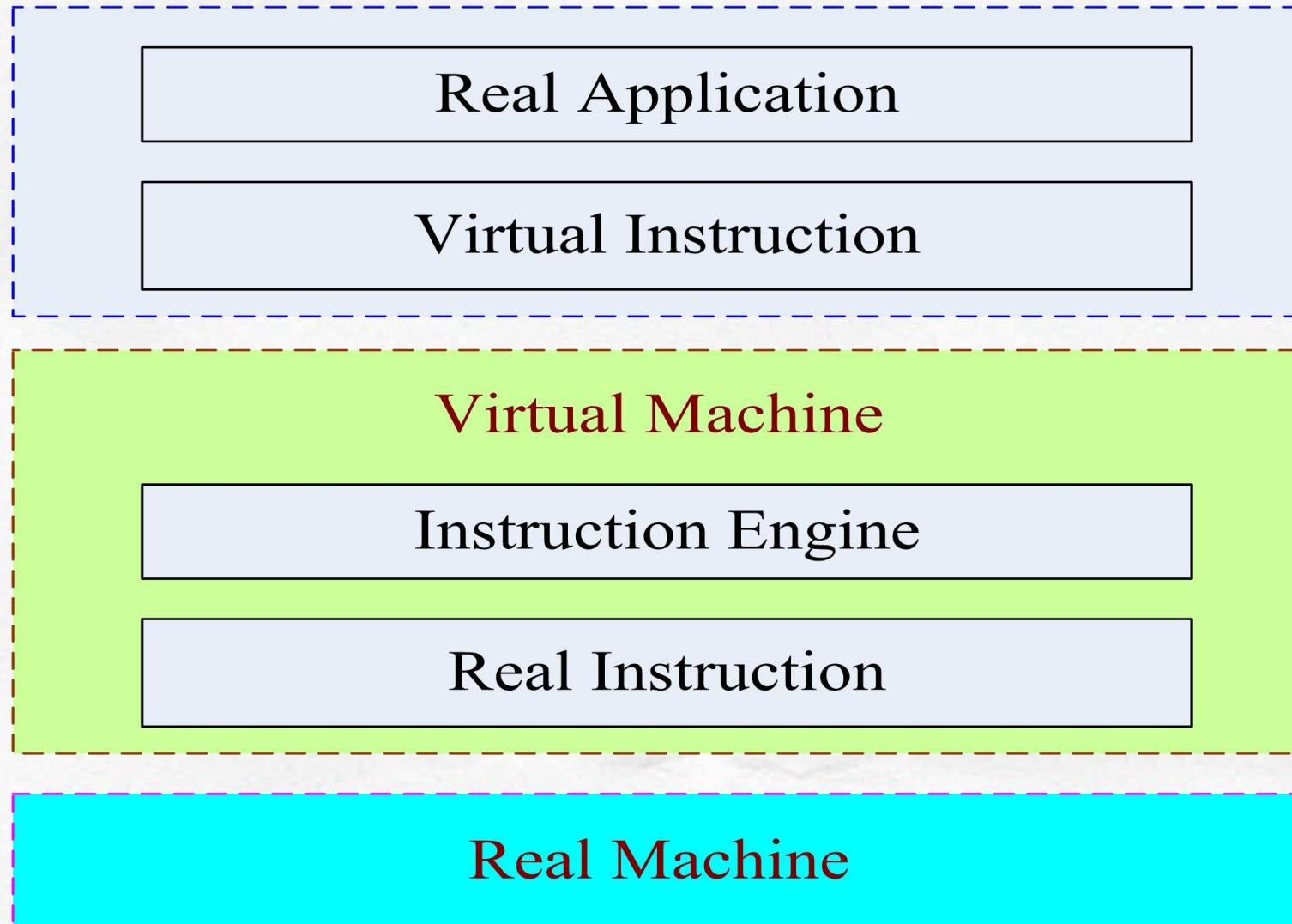


虚拟机体系架构

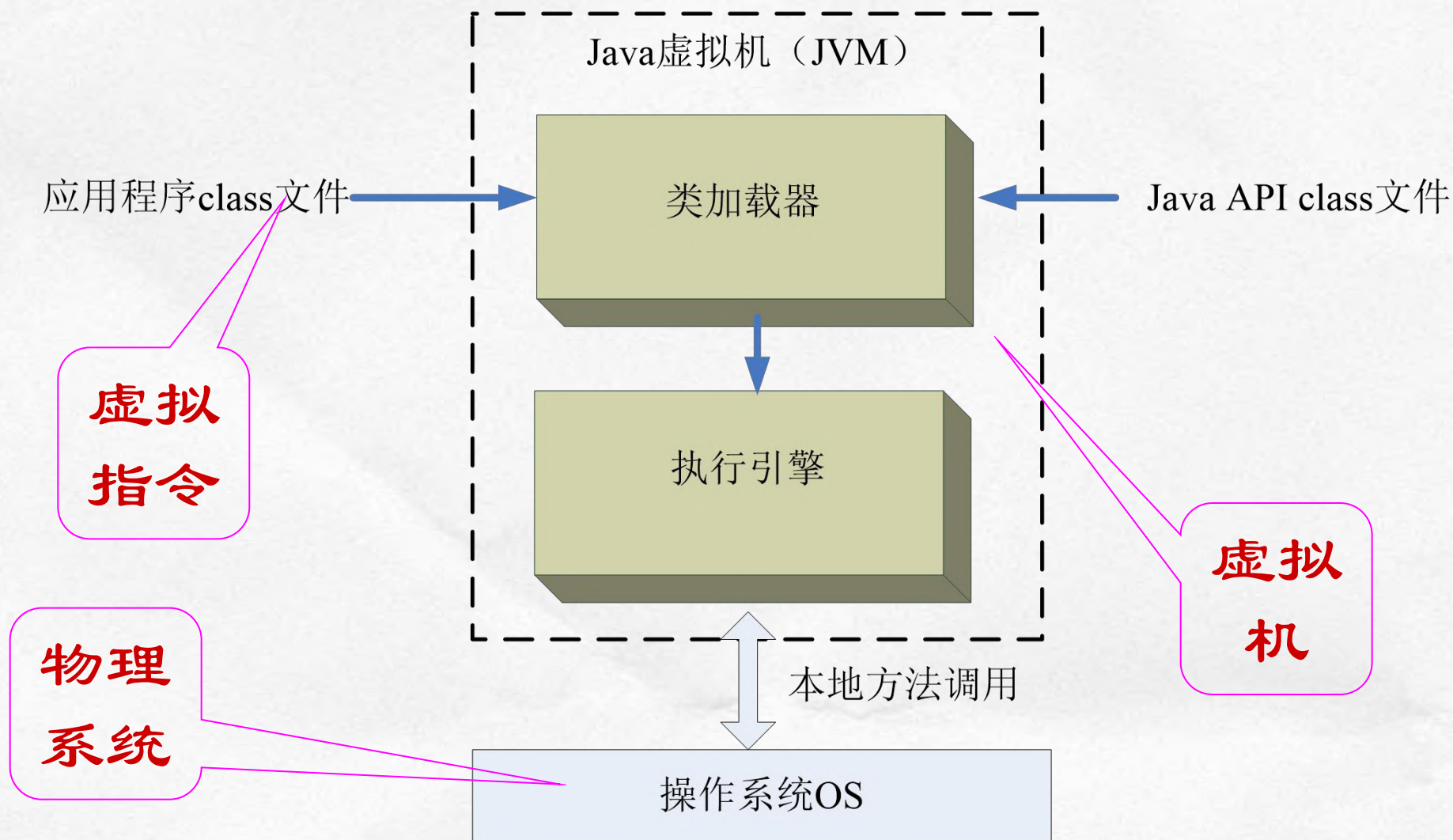
以牺牲运行效率为代价，优化应用的可移植性。

- 一个抽象计算机，有自己的抽象指令
- 解释执行抽象指令（应用），在实际计算机上运行
- 应用可在新环境下运行，不重新编码、编译
- Basic、Java

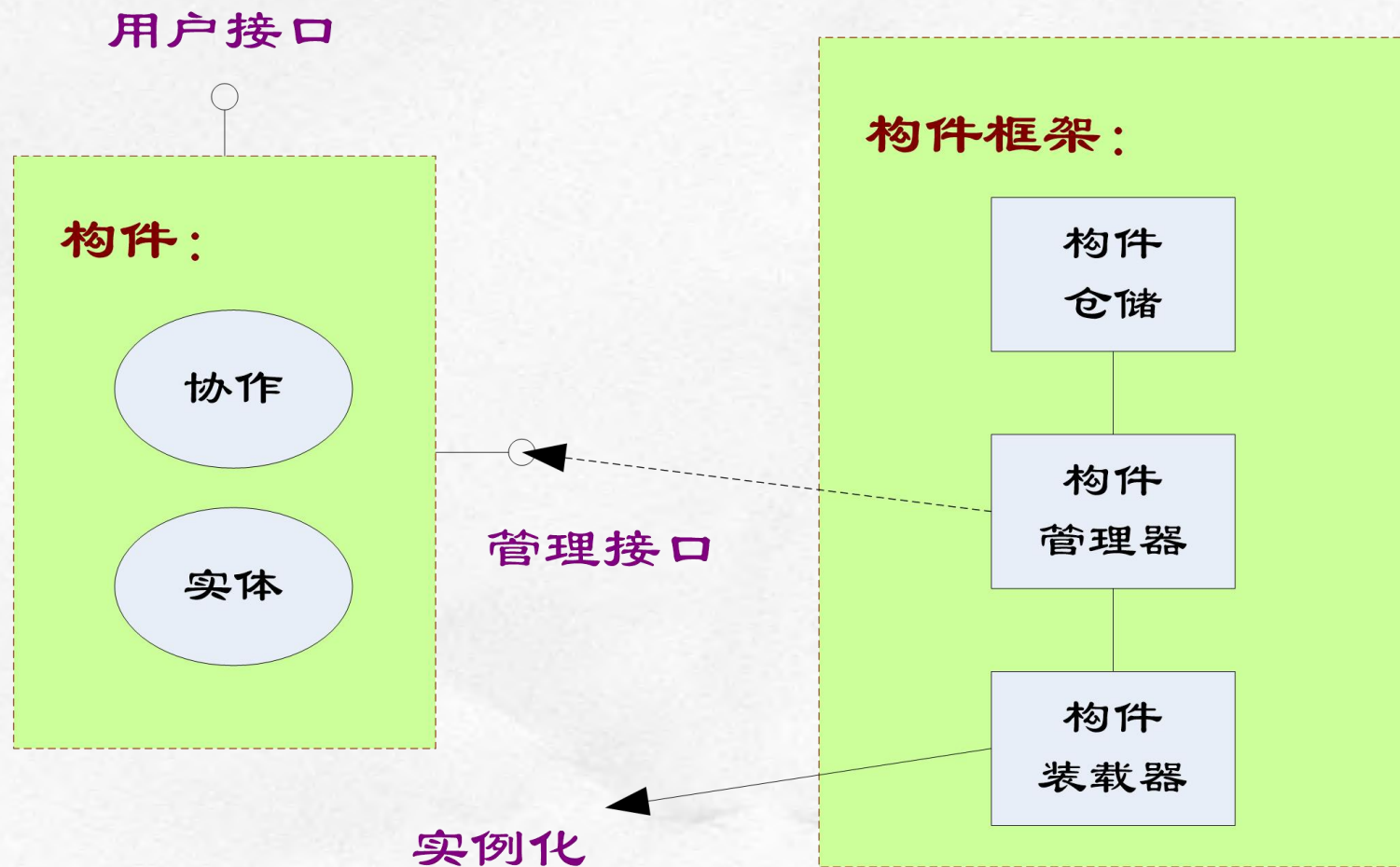
虚拟机体系架构 (2)



虚拟机体系架构 (3) : JVM架构

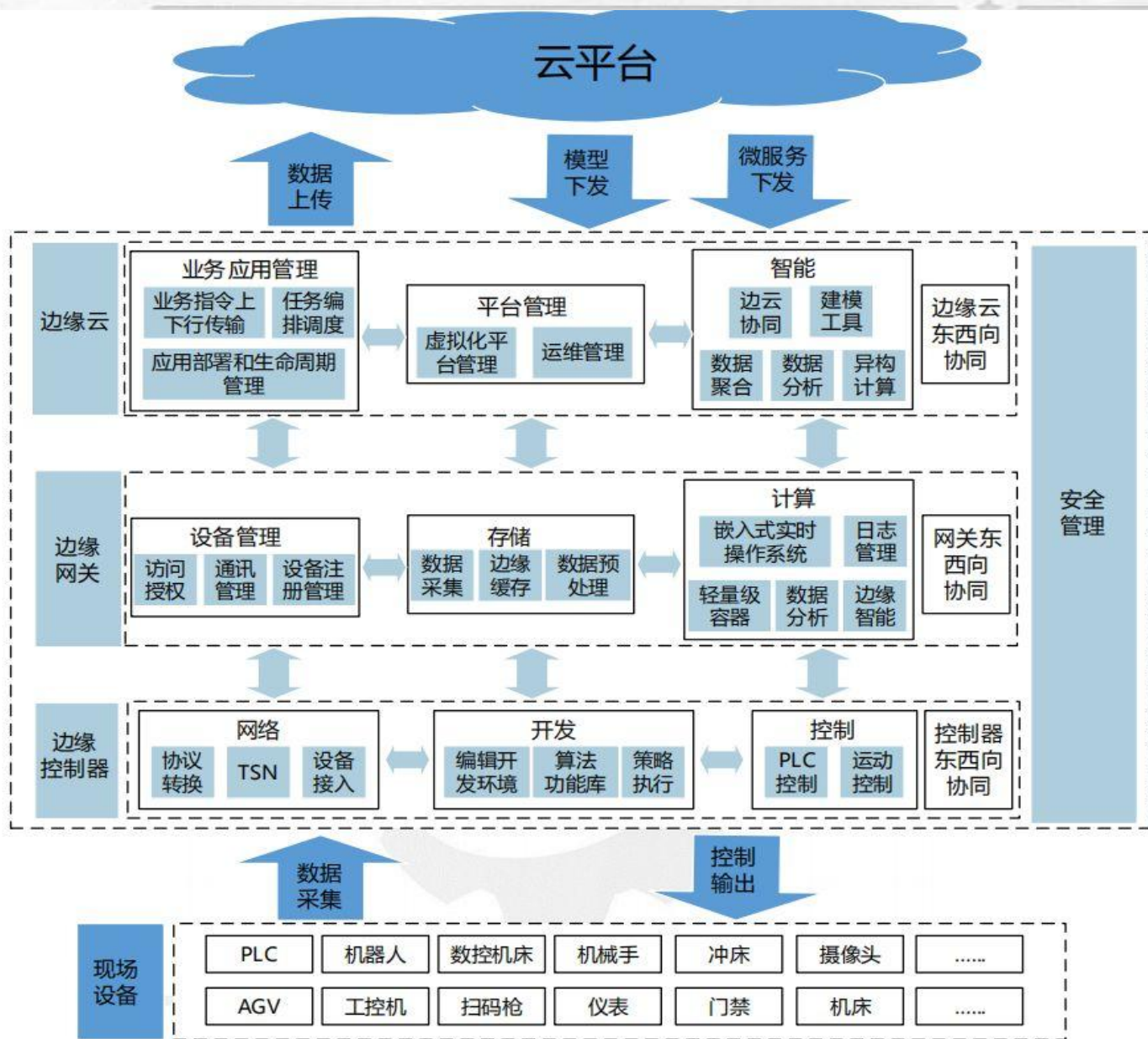


基于构件的架构



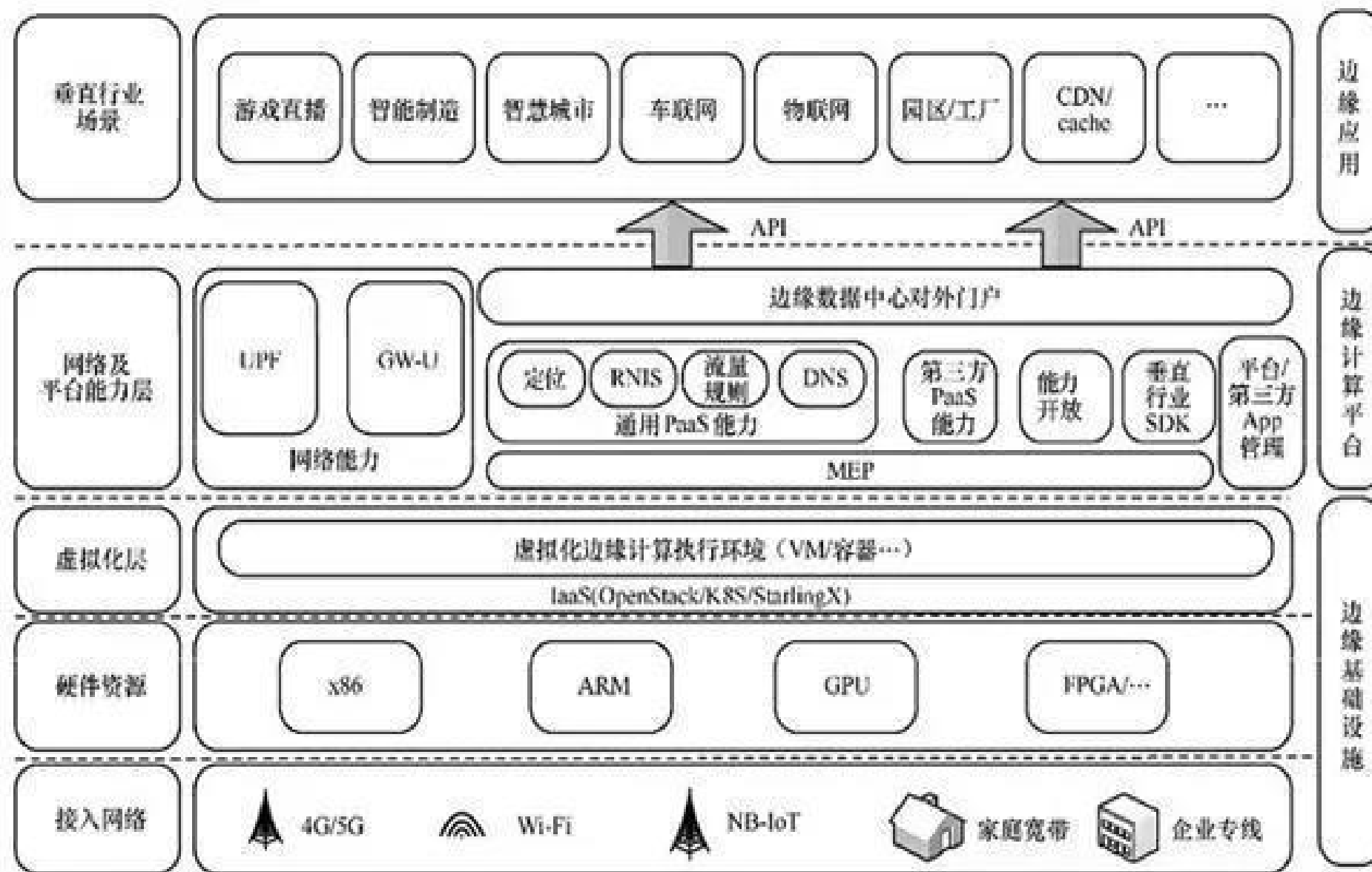
基于构件的体系结构模型

边缘计算体系架构

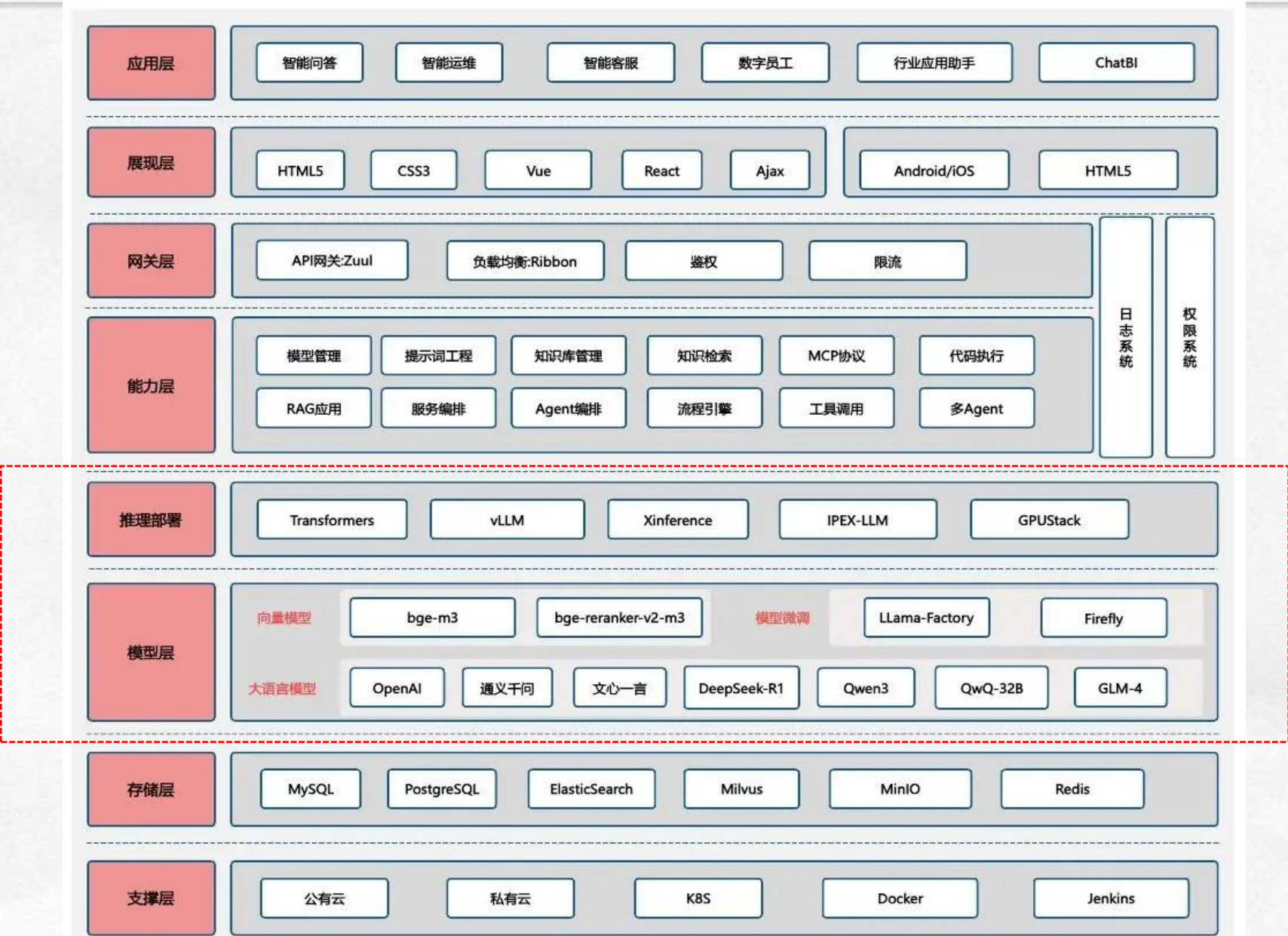


- **边缘计算**：一种将计算资源部署在网络边缘（即离数据源更近的地方）的技术
- **雾计算**：一种将计算资源部署在网络边缘（即离数据源更近的地方）的技术
- **云计算**：一种通过互联网为用户提供按需计算服务的模式

边缘计算体系架构 (2)



基于AI大模型的架构



MVC模式

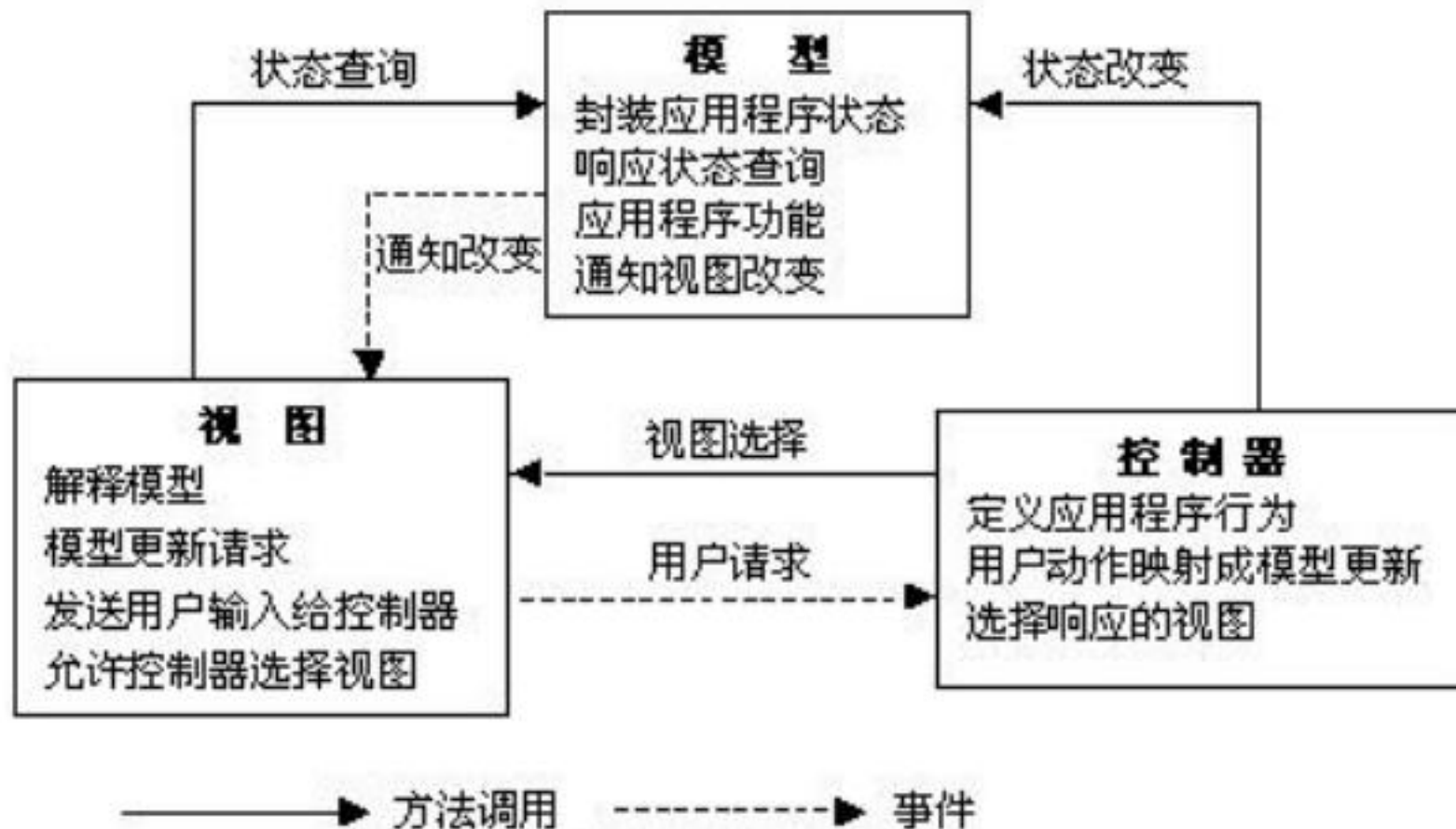


图1 MVC组件类型的关系和功能

模型驱动的 软件开发

传统设计方法

- 当面对一个新的业务需求时，总是从先建立数据表等结构开始
- 面向功能的结构化设计方法
- 面向数据流的结构化设计方法
- 面向对象设计方法

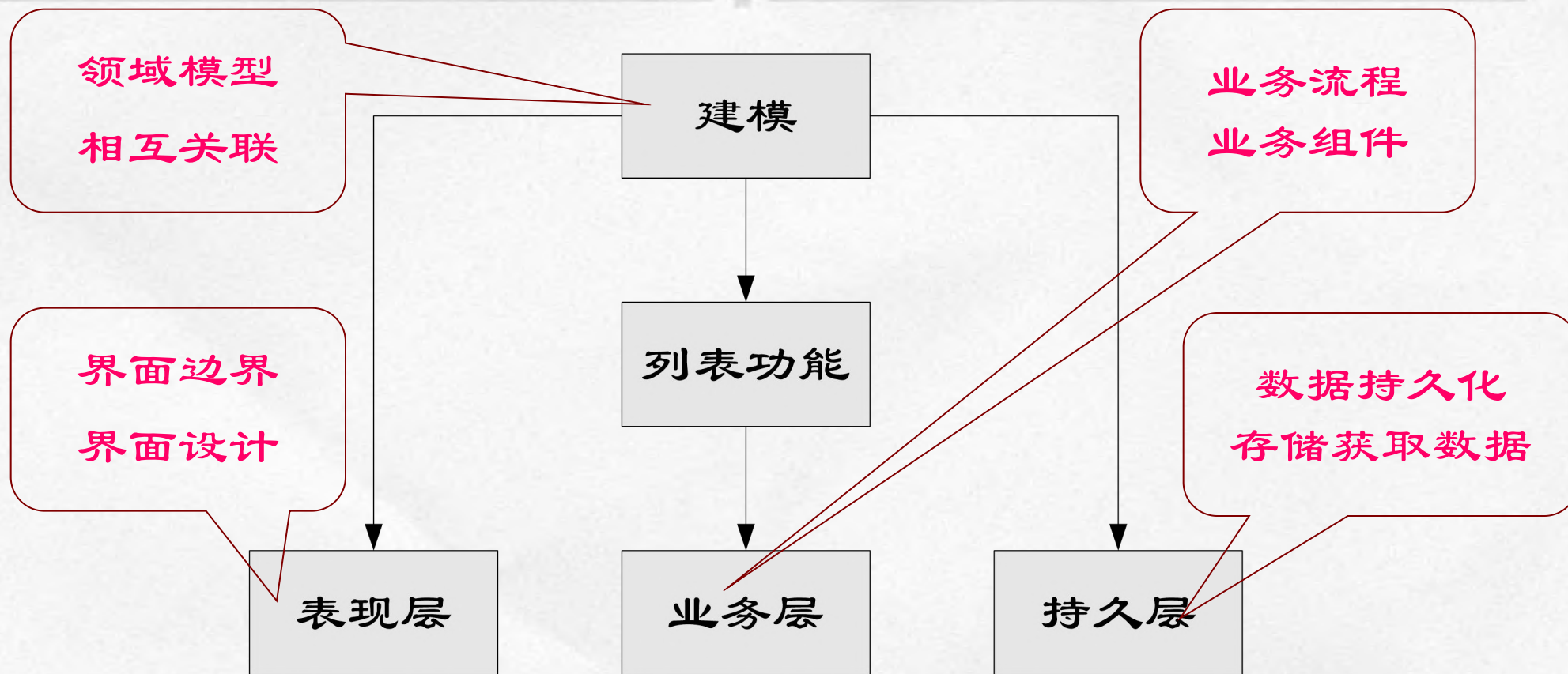
存在问题

- 系统的状态无法统一设计，导致每个程序员都可以直接操控系统的状态——整个系统状态运行混乱
- 容易将实体表和关系混合，造成分析者视觉混乱，无法正确提炼系统核心实体
- 软件系统的编码架构以数据库为中心：沿用过程化编程风格，难于维护和拓展
- 系统负载集中在数据库服务器端：大型机集中式计算模式，非分布式计算模式
- 容易丧失多层结构引以为豪的中间层，回复到过去的两层结构，不利于软件重用。

模型驱动的软件架构

- **起源**：OMG定义的一个软件开发框架，基于UML
- **MDA概念**：以建模行为驱动整个软件开发过程
- **核心思想**：将业务分析与业务实现分离，保持业务逻辑对实现技术和平台的独立性，提高软件质量和可重用度
- **意义**：提供了一种途径（通过相关的工具）来规范化一个平台独立的系统、规范化平台、为系统选择一个特定的实现平台，并且把系统规范转换到特定的实现平台

模式驱动的设计过程

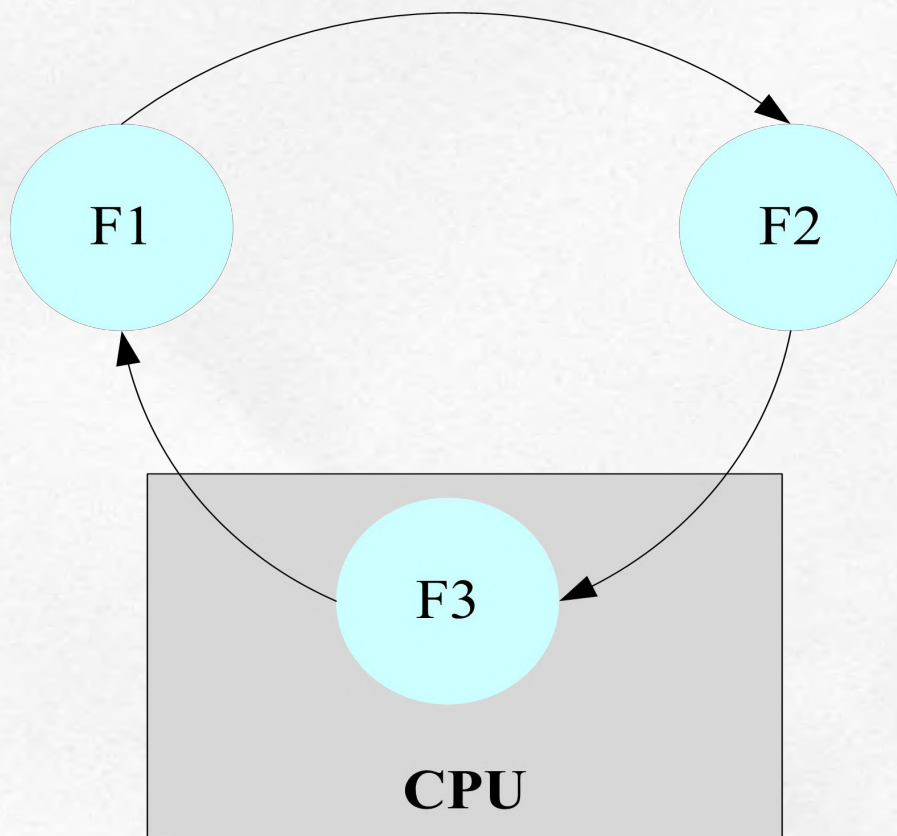


确立领域模型及其之间关系；定义业务功能；由程序员分别实现表现层、业务服务层和持久层

软件体系结构

嵌入式实时模型

轮询模型



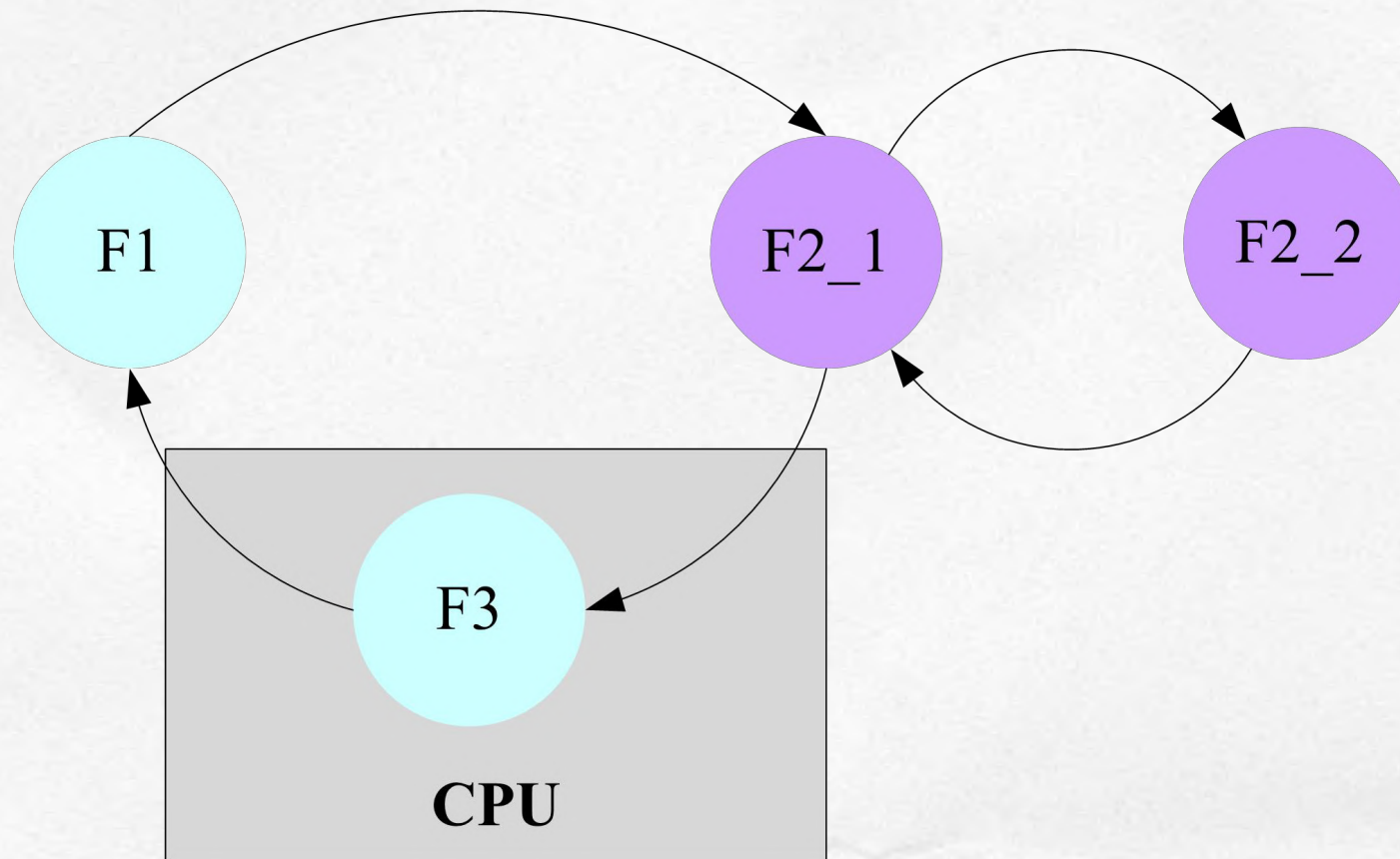
注：F_i代表函数名 (i=1, 2...)

轮询过程

```
initialize( );  
while (TRUE){  
    if (condition1) {F1();}  
    if (condition2) {F2();}  
    if (condition3) {F3();}  
    ...  
}
```

轮询程序框架

轮询 (2) —— 子轮询结构

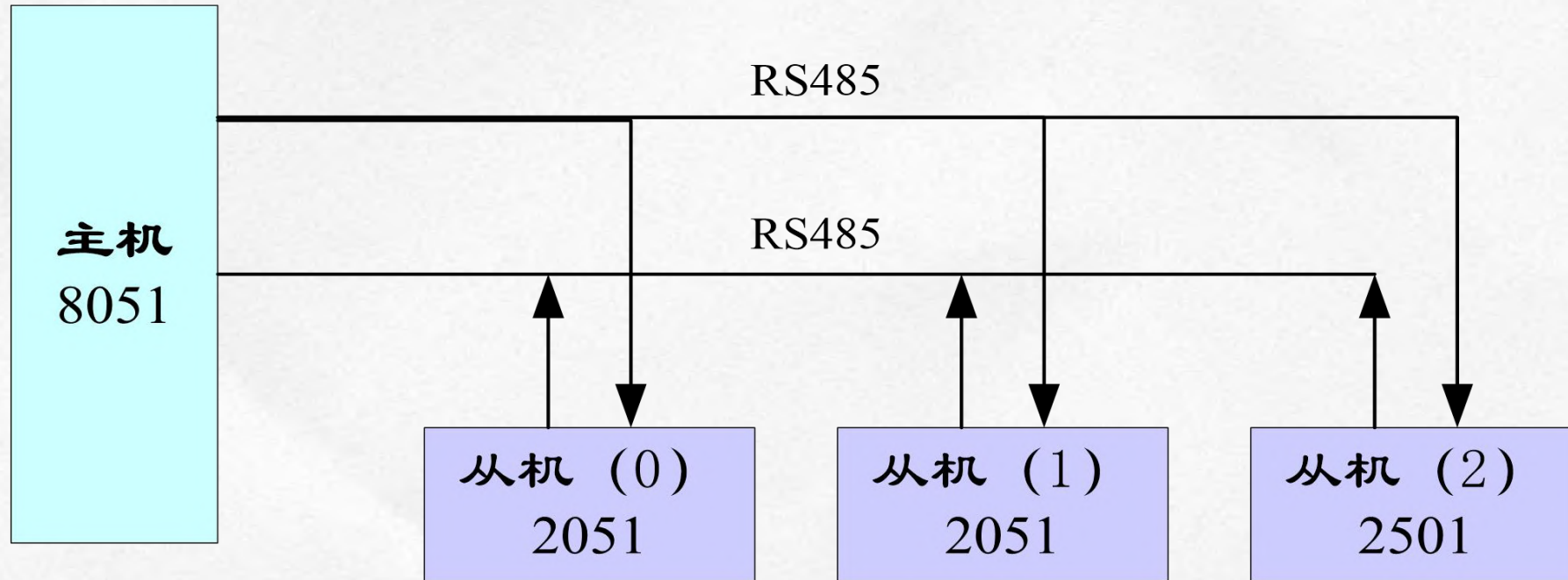


子轮询结构

轮询 (2) —— 子轮询程序

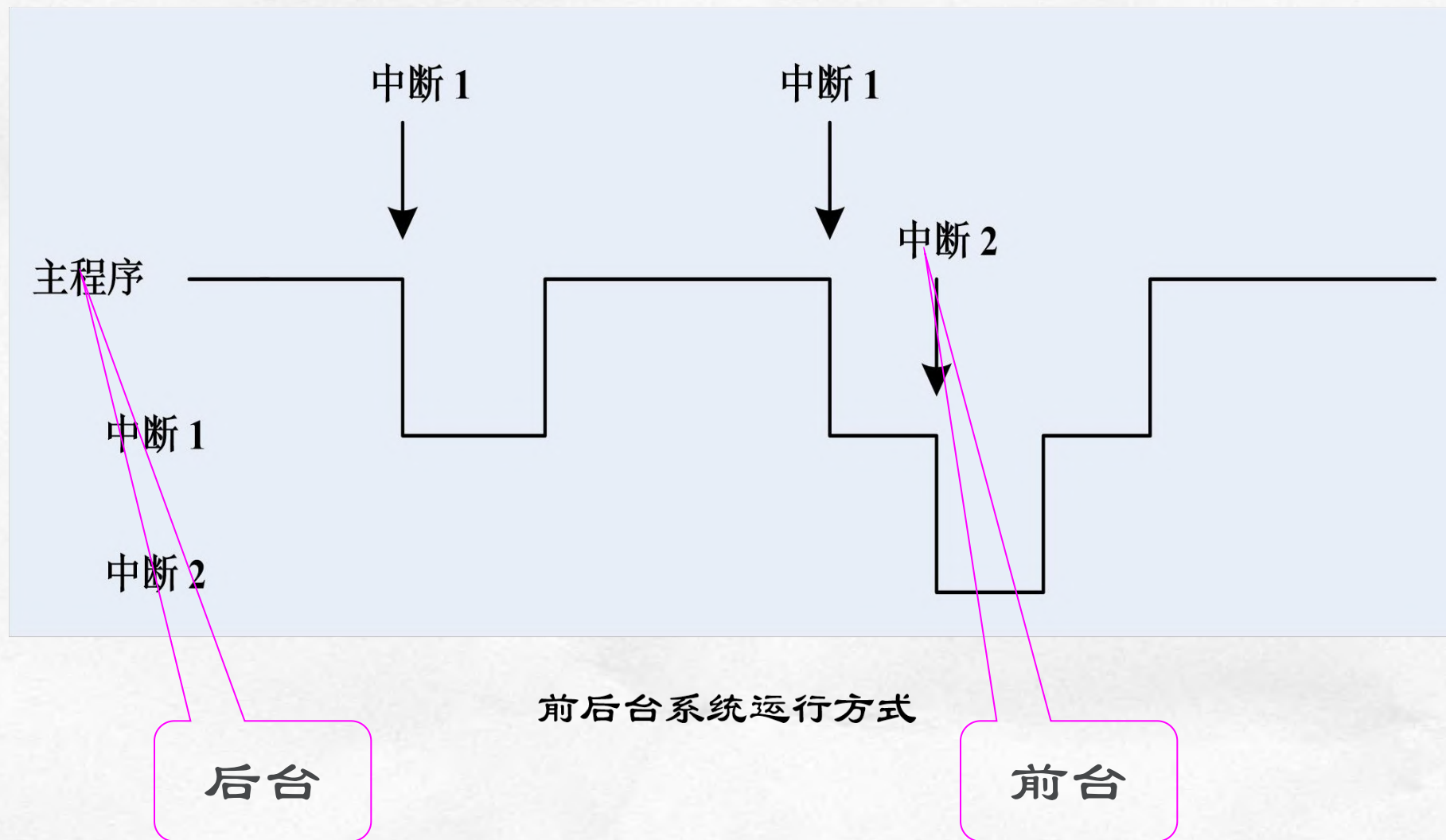
```
print_msg(msg)
char *msg;
{
    while (*msg!=END_CHAR){
        if (READY_FLAG & input(PRINTER_PORT)){
            output(PRINTER_DATA,*MSG++);
        }else{
            poll_others( );
        }
    }
}
```


轮询 (3) —— 典型结构及应用示意

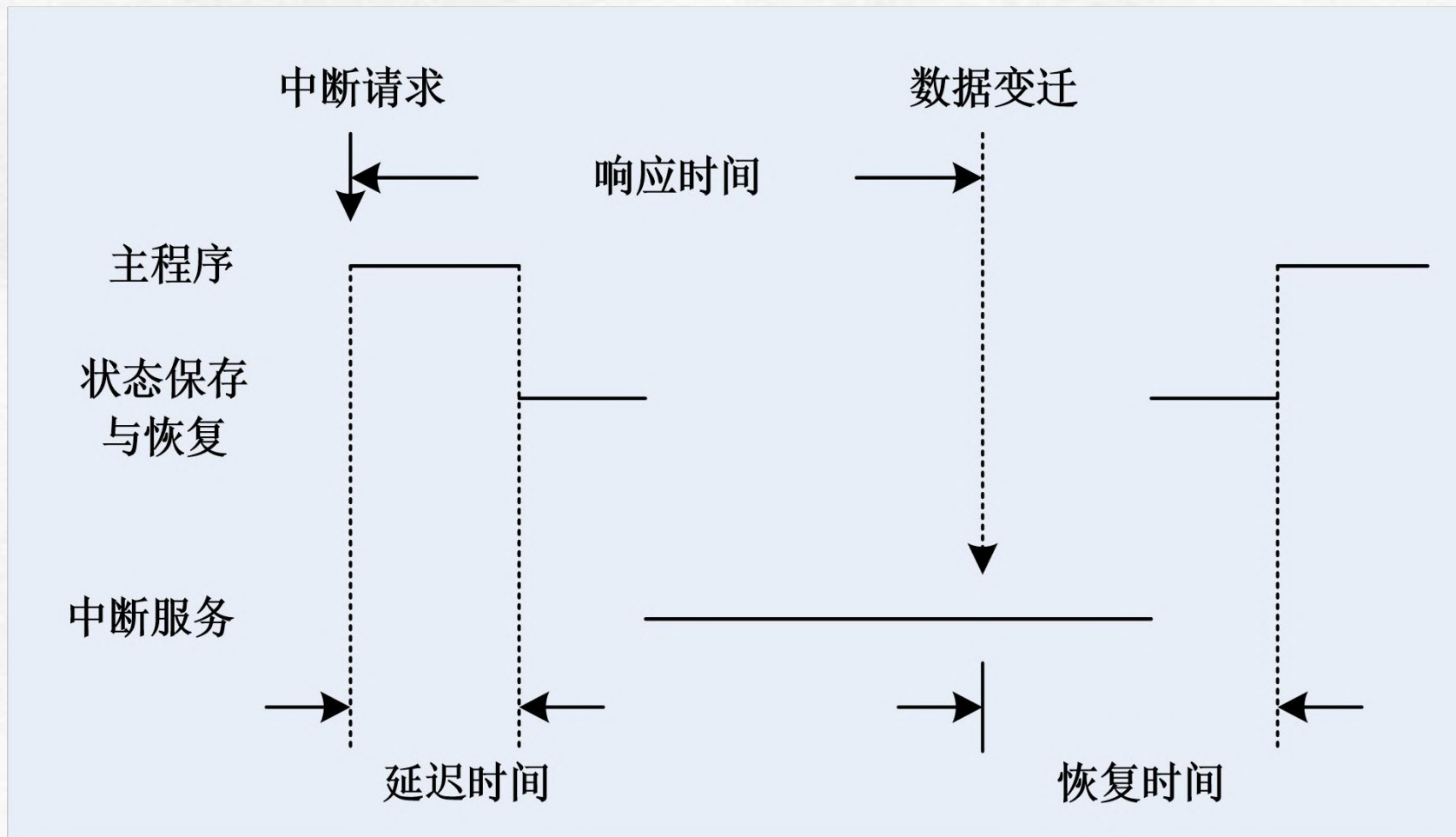


- **典型应用** 哲学家问题；无线传感信息采集；分时系统；信号（如雷达信号）采集系统；网络服务应答；....

前后台模型 —— 中断模型



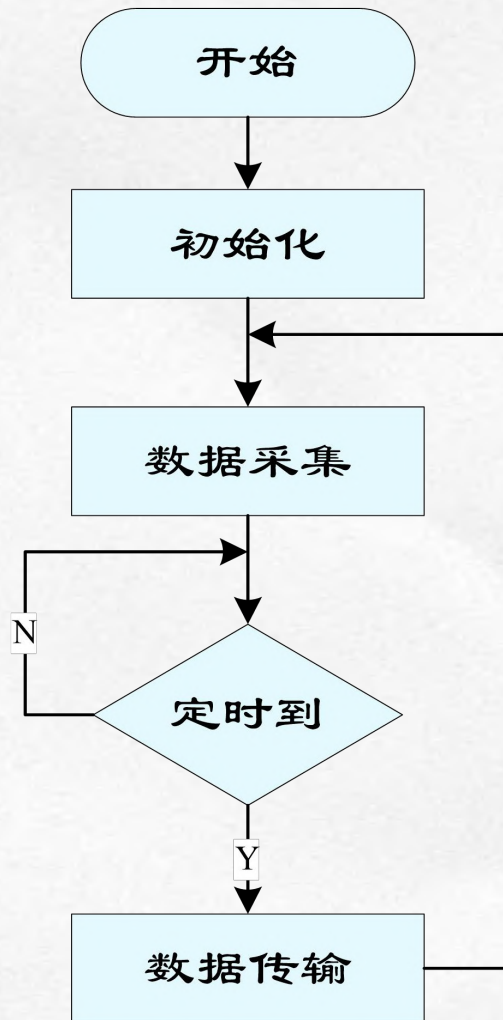
前后台模型 (2) : 中断的执行



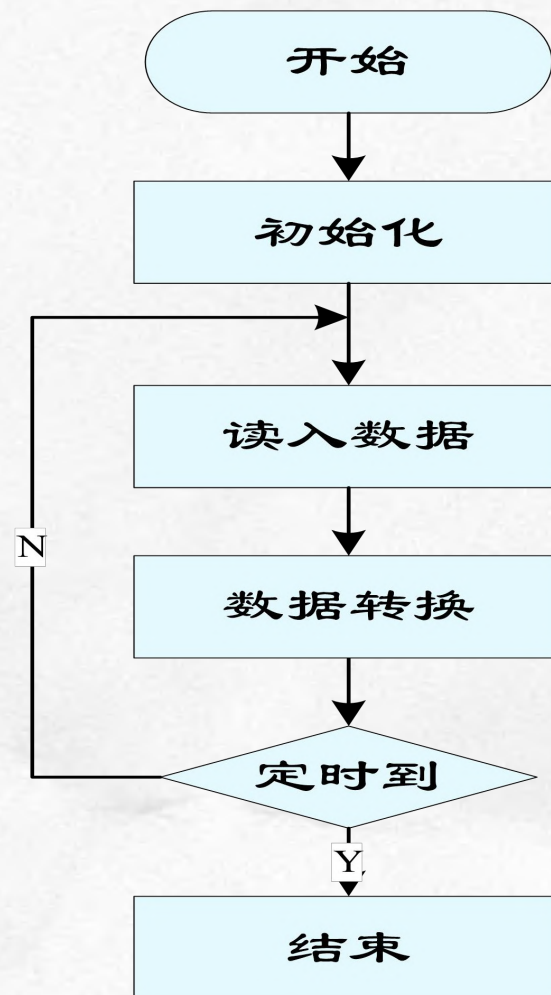
中断执行的刻画

前后台模型 (3) —— 典型系统结构

后台：采集主程序



前台：中断程序



● 典型应用

信号（如雷达信号）采集系统；开启节能模式的微控制系统；酒店预订系统、....

前后台模型 (4) : 重要参数

- 性能指标：响应时间

中断延迟时间

吞吐量

- 前后台交互：同步信号

数据交互

中断

- **中断** 指一个必须立即处理的情况时，CPU暂时停止当前程序的执行转而执行处理新情况的程序和执行过程
 - 一种硬件机制，芯片必须带该机制
- **中断服务** 一段代码或程序，中断发生时CPU新执行的内容
- **相关概念**
 - **中断注册** 使中断服务程序有效
 - **中断优先级** 中断服务程序的紧急程度
 - **中断屏蔽** 确保重要程序不被中断，以免引发错误
 - **中断源** 可能引起中断的设备或故障

中断 —— 处理过程

- (1) 中断CPU当前执行
- (2) 保护被中断进程现场。
- (3) 分析中断原因，转去相应的中断处理程序。

保存什么？

➤ 在多个中断请求同时发生时，处理优先级最高的中断源发出的中断请求。

如何得到处理程序？

- (4) 执行中断处理程序
- (5) 恢复被中断进程的现场
- (6) CPU继续执行原来被中断的进程。

从哪开始？

中断 —— 现场保存与恢复

- 保存/恢复内容 寄存器

链接寄存器，存放

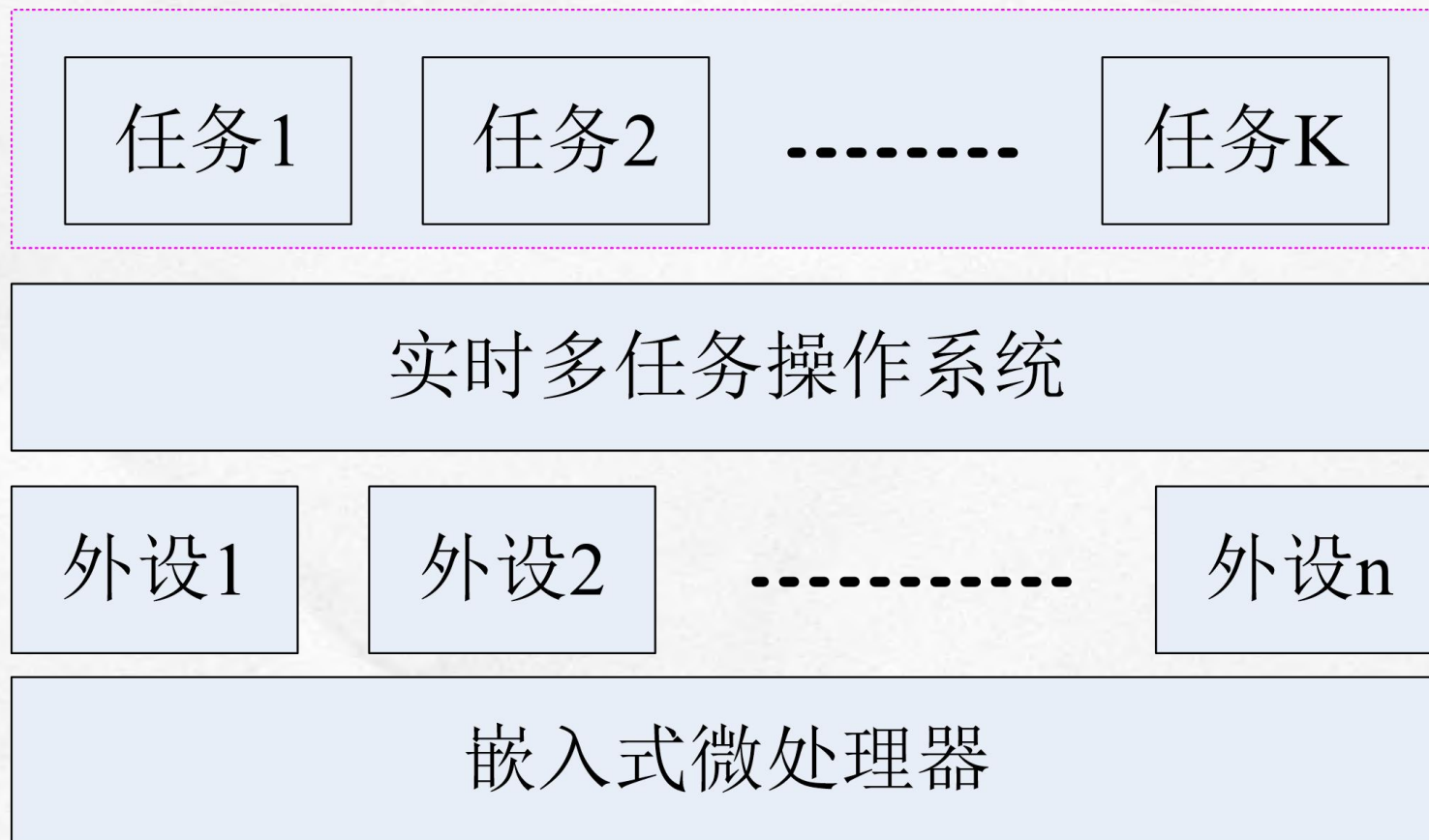
子程序返回地址

; 现场保存: r0-r12、LR、CPSR		
sub lr, lr, #4	; 由IRQ模式计算返回地址	(1)
stmfd sp!, {r0-r12, lr}	; 保存寄存器和pc	(2)
mrs r0, cpsr		(3)
stmfd sp!, {r0}	; 保存CPSR	(4)
; 跳转执行相应的中断服务程序		
ldr r0, =INTOFFSET		(5)
ldr r0, [r0]		(6)
ldr r1, =HandlerEINT0		(7)
add r1, r1, r0, lsl #2	; R0=R1+R2*4,	(8)
ldr r1, [r1]		(9)
mov lr, pc	; ISR完成后的返回处	(10)
mov pc, r1	; 跳转到ISR	(11)
; 恢复现场: r0-r12、LR、CPSR		
ldmfd sp!, {r0}		(12)
mrs spsr_cxsf, r0	; 恢复CPSR	(13)
ldmfd sp!, {r0-r12, lr}	; 恢复寄存器和pc	(14)
movs pc, lr	; 返回被中断程序	(15)

实时多任务模型

- **任务 (Task)** 一个程序运行的实体；资源拥有的基本单位；系统调度的基本单元（若不含线程）
- **实时任务 (Real-time Task)** 一个带有实时性能约束的任务
- **实时操作系统 (RTOS)** 构架系统时从系统中抽象出一个便于上层应用开发的平台，为支持实时多任务应用

实时多任务模型 (2)



实时多任务系统结构

实时多任务模型 (3) : 示例

Application

APP 1

APP 2

.....

APP n

M/W

CORBA

JAVA

DCOM

.....

OS

TCP/IP

File
System

Kernel

Power
Manage

GUI

Driver

BSP

FLASH

RTC

SPI

PPI

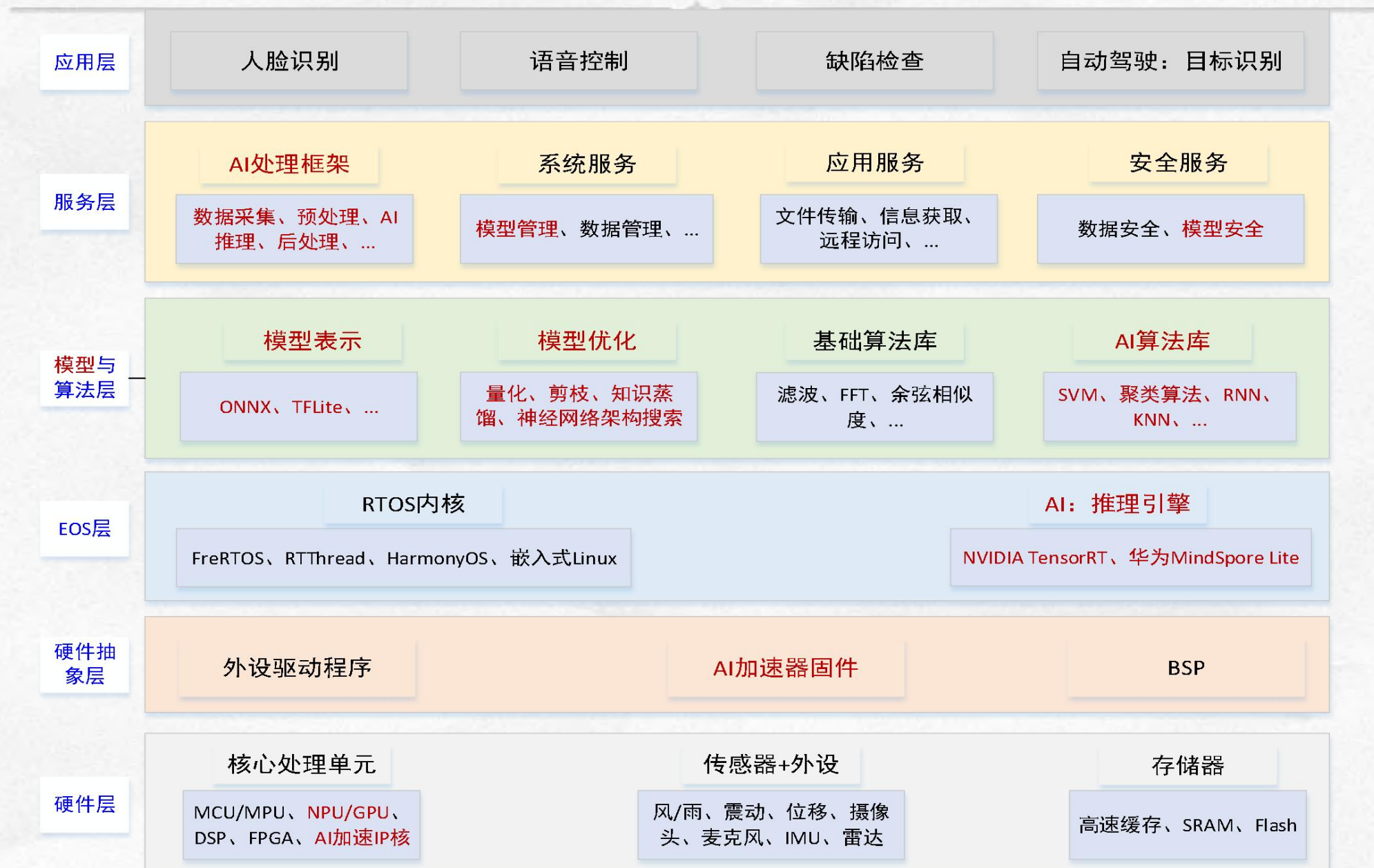
LCD

CAN

.....

H/W

基于AI的嵌入式系统模型



思考题

1. 前后台系统在轮询系统的基础上做了什么改变？
其运行方式是什么？它适用于什么嵌入式应用？
2. 结合代码分析：IRQ中断发生后，ARM9
S3C2440如何保护做现场保护？需要保持哪些信息？
3. 加入AI后，嵌入式系统体系架构有哪些改变？

The slide features a light gray background with a faint, abstract pattern. A thin horizontal line spans the width of the slide, passing behind the central text. Three teal-colored circles are positioned around the slide: one on the left edge, one on the right edge, and a larger one at the top center.

谢谢！